



Experiment 3: Perceptron vs Multilayer Perceptron (A/B Experiment) with Hyperparameter Tuning

NAME : SREENETHI G S REGISTER NO : 3122237001052

Aim

To implement and compare a Single-Layer Perceptron (PLA) and a Multilayer Perceptron (MLP) for handwritten character recognition, using systematic hyperparameter tuning to optimize performance.

Objective

To implement and compare the performance of: • Model A: Single-Layer Perceptron Learning Algorithm (PLA). • Model B: Multilayer Perceptron (MLP) with hidden layers and nonlinear activations. Students must select and justify hyperparameters such as activation functions, cost functions, optimizers, learning rates, number of hidden layers, and batch sizes through systematic tuning.

In[6]:

```
# Import necessary libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from PIL import Image
import os
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler, label_binarize
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset
import warnings
warnings.filterwarnings('ignore')

# Mount Google Drive (if running in Colab)
from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

1. LOAD PREPROCESS AND NORMALISE DATASET

In[3]:

```
# Load the dataset information
data_path = '/content/drive/MyDrive/ml5/' # Update this path
```

```

csv_file=pd.read_csv(data_path+'english.csv')# Update with your CSV filename

# Display first few rows
print(csv_file.head())

# Check class distribution
plt.figure(figsize=(12,6))
sns.countplot(x='label',data=csv_file)
plt.title('Class Distribution')
plt.xticks(rotation=90)
plt.show()

# Function to load and preprocess images
defload_images(image_paths,img_size=(32,32)):
    images=[]
    forimg_pathinimage_paths:
        # Construct full path
        full_path=data_path+img_path

        # Open and resize image
        img=Image.open(full_path).convert('L')# Convert to grayscale
        img=img.resize(img_size)

        # Convert to numpy array and normalize
        img_array=np.array(img)/255.0

        # Flatten the image
        img_flattened=img_array.flatten()

        images.append(img_flattened)

    returnnp.array(images)

# Load and preprocess images
print("Loading images...")
X=load_images(csv_file['image'])
y=csv_file['label'].values

# Encode labels
label_encoder=LabelEncoder()
y_encoded=label_encoder.fit_transform(y)

# Split the data
X_train,X_test,y_train,y_test=train_test_split(
    X,y_encoded,test_size=0.2,random_state=42,stratify=y_encoded
)

X_train,X_val,y_train,y_val=train_test_split(
    X_train,y_train,test_size=0.2,random_state=42,stratify=y_train
)

print(f"Training set:{X_train.shape},{y_train.shape}")
print(f"Validation set:{X_val.shape},{y_val.shape}")

```

```

print(f"Test set:{X_test.shape},{y_test.shape}")

# Normalize the data
scaler=StandardScaler()
X_train=scaler.fit_transform(X_train)
X_val=scaler.transform(X_val)
X_test=scaler.transform(X_test)

# Convert to PyTorch tensors for MLP
X_train_tensor=torch.FloatTensor(X_train)
y_train_tensor=torch.LongTensor(y_train)
X_val_tensor=torch.FloatTensor(X_val)
y_val_tensor=torch.LongTensor(y_val)
X_test_tensor=torch.FloatTensor(X_test)
y_test_tensor=torch.LongTensor(y_test)

# Create DataLoader for batch processing
train_dataset=TensorDataset(X_train_tensor,y_train_tensor)
val_dataset=TensorDataset(X_val_tensor,y_val_tensor)
test_dataset=TensorDataset(X_test_tensor,y_test_tensor)

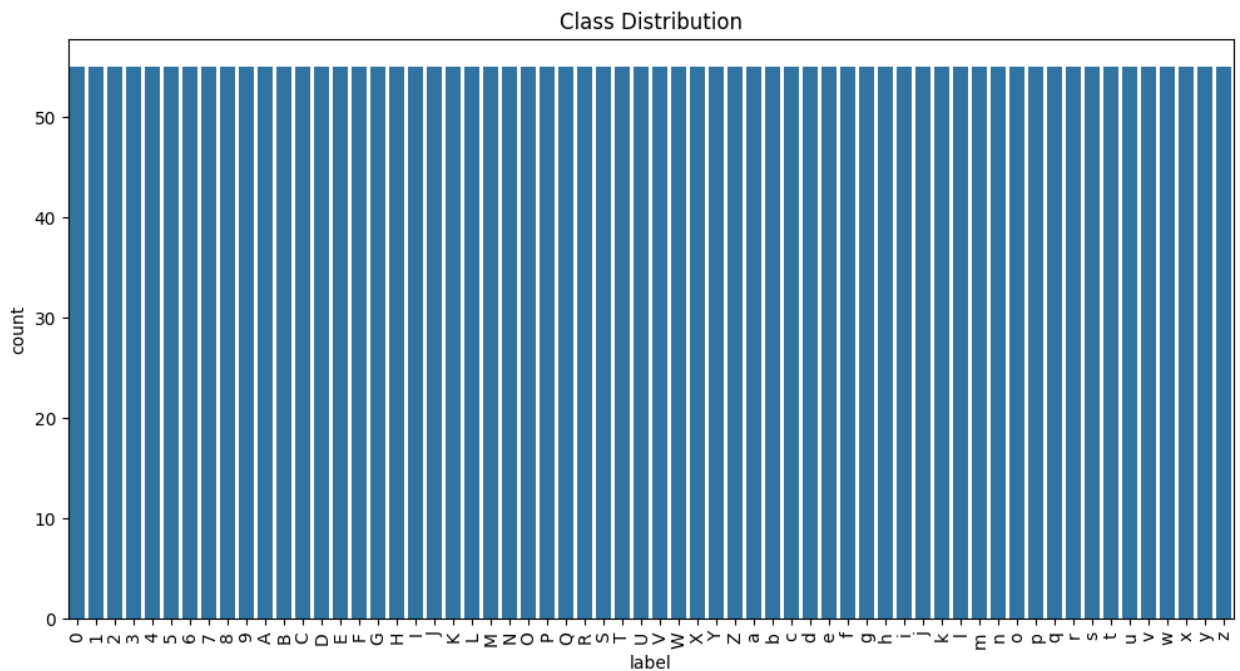
```

image label

```

0 Img/img001-001.png 0
1 Img/img001-002.png 0
2 Img/img001-003.png 0
3 Img/img001-004.png 0
4 Img/img001-005.png 0

```



Loading images...

Training set: (2182, 1024), (2182,)

Validation set: (546, 1024), (546,)

Test set: (682, 1024), (682,)

**2. Perceptron Learning Algorithm (PLA) Implementation and Results

In[4]:

```
# Perceptron Learning Algorithm (PLA) Implementation
class PLA:
    def __init__(self, input_size, num_classes, learning_rate=0.01):
        self.weights = np.zeros((num_classes, input_size))
        self.bias = np.zeros(num_classes)
        self.learning_rate = learning_rate
        self.num_classes = num_classes
        self.train_errors = [] # To track training errors per epoch

    def step_activation(self, x):
        return np.where(x >= 0, 1, 0)

    def predict(self, x):
        # One-vs-all approach for multiclass
        outputs = np.dot(self.weights, x) + self.bias
        return np.argmax(outputs)

    def predict_proba(self, x):
        # For ROC curve, we need probability estimates
        outputs = np.dot(self.weights, x) + self.bias
        # Simple softmax-like conversion (not exactly probabilities but works for ROC)
        exp_outputs = np.exp(outputs - np.max(outputs)) # For numerical stability
        return exp_outputs / np.sum(exp_outputs)

    def train(self, X, y, epochs=100):
        n_samples = X.shape[0]

        for epoch in range(epochs):
            errors = 0
            for i in range(n_samples):
                x_i = X[i]
                y_true = y[i]

                # Prediction
                y_pred = self.predict(x_i)

                # Update weights if prediction is wrong
                if y_pred != y_true:
                    # Decrease score for wrong class
                    self.weights[y_pred] -= self.learning_rate * x_i
                    self.bias[y_pred] -= self.learning_rate

                    # Increase score for correct class
                    self.weights[y_true] += self.learning_rate * x_i
                    self.bias[y_true] += self.learning_rate

            errors += 1
```

```

        # Record error rate for this epoch
        error_rate=errors/n_samples
        self.train_errors.append(error_rate)

    # Print progress
    if(epoch+1)%10==0:
        print(f"Epoch{epoch+1}/{epochs}, Error Rate:{error_rate:.4f}")

    # Early stopping if no errors
    if errors==0:
        print(f"Converged at epoch{epoch+1}")
        break

    def evaluate(self,X,y):
        n_samples=X.shape[0]
        y_pred=[]
        y_proba=[]

        for i in range(n_samples):
            pred=self.predict(X[i])
            y_pred.append(pred)
            y_proba.append(self.predict_proba(X[i]))

        return np.array(y_pred),np.array(y_proba)

# Initialize and train PLA
print("Training PLA...")
input_size=X_train.shape[1]
num_classes=len(np.unique(y_train))

pla=PLA(input_size,num_classes,learning_rate=0.01)
pla.train(X_train,y_train,epochs=100)

# Plot PLA training convergence
plt.figure(figsize=(10,6))
plt.plot(pla.train_errors)
plt.title('PLA Training Error Convergence')
plt.xlabel('Epoch')
plt.ylabel('Error Rate')
plt.grid(True)
plt.show()

# Evaluate PLA
y_pred_pla,y_proba_pla=pla.evaluate(X_test,y_test)
pla_accuracy=accuracy_score(y_test,y_pred_pla)
pla_precision=precision_score(y_test,y_pred_pla,average='weighted',zero_division=0)
pla_recall=recall_score(y_test,y_pred_pla,average='weighted',zero_division=0)
pla_f1=f1_score(y_test,y_pred_pla,average='weighted',zero_division=0)

print(f"PLA Test Accuracy:{pla_accuracy:.4f}")
print(f"PLA Test Precision:{pla_precision:.4f}")
print(f"PLA Test Recall:{pla_recall:.4f}")
print(f"PLA Test F1-Score:{pla_f1:.4f}")

```

```

# Confusion matrix for PLA
plt.figure(figsize=(10,8))
cm_pla=confusion_matrix(y_test,y_pred_pla)
sns.heatmap(cm_pla,annot=False,fmt='d',cmap='Blues')
plt.title('PLA Confusion Matrix')
plt.ylabel('True Label')
plt.xlabel('Predicted Label')
plt.show()

# Classification report for PLA
class_names=label_encoder.classes_
print("Report for PLA:")
print(classification_report(y_test,y_pred_pla,target_names=class_names,zero_division

# Visualize some PLA predictions
defvisualize_pla_predictions(pla_model,X,y,label_encoder,num_samples=10):
    plt.figure(figsize=(15,6))
    indices=np.random.choice(len(X),num_samples,replace=False)

    fori,idxinenumerate(indices):
        plt.subplot(2,5,i+1)
        # Reshape flattened image to 2D (assuming original was 32x32)
        img=X[idx].reshape(32,32)
        plt.imshow(img,cmap='gray')

        true_label=label_encoder.inverse_transform([y[idx]])[0]
        pred_label=label_encoder.inverse_transform([pla_model.predict(X[idx])])[

        title_color='green'iftrue_label==pred_labelelse'red'
        plt.title(f"True:{true_label}:{pred_label}",color=title_color)
        plt.axis('off')

    plt.tight_layout()
    plt.show()

visualize_pla_predictions(pla,X_test,y_test,label_encoder)

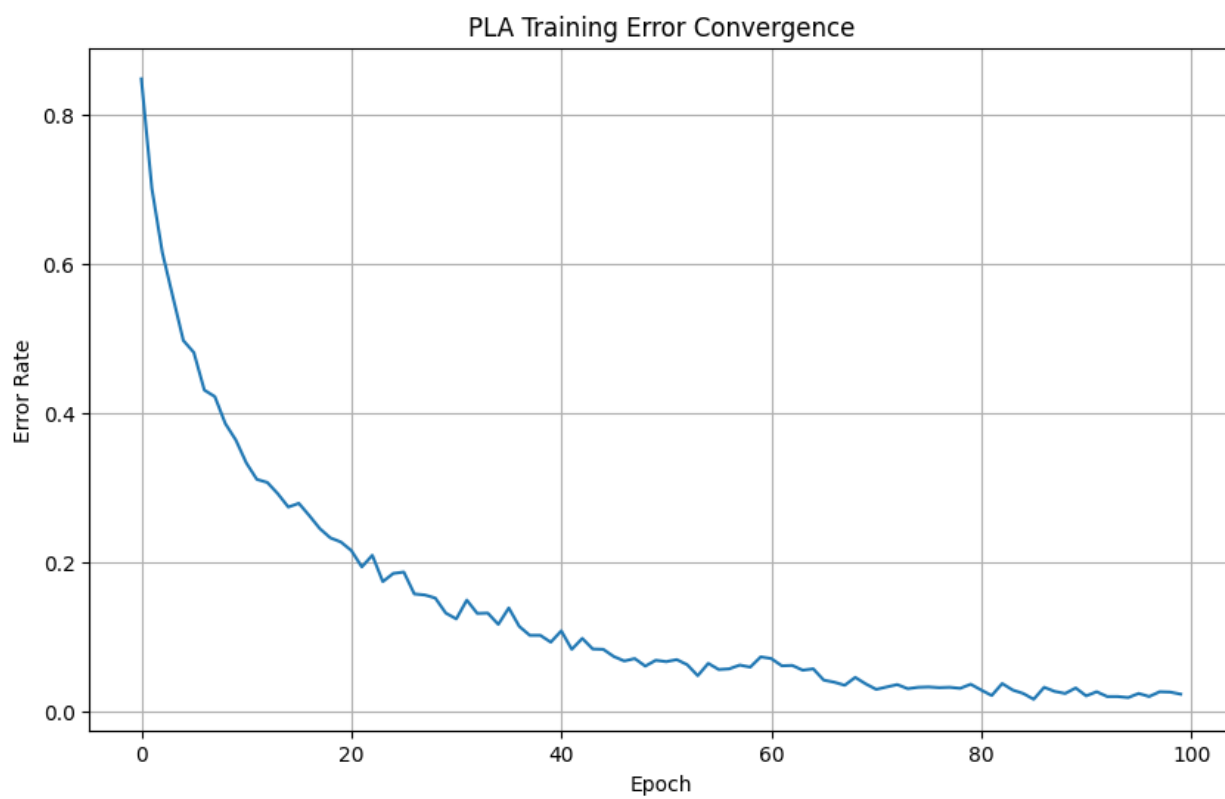
```

Training PLA...

```

Epoch 10/100, Error Rate: 0.3639
Epoch 20/100, Error Rate: 0.2273
Epoch 30/100, Error Rate: 0.1320
Epoch 40/100, Error Rate: 0.0930
Epoch 50/100, Error Rate: 0.0687
Epoch 60/100, Error Rate: 0.0733
Epoch 70/100, Error Rate: 0.0371
Epoch 80/100, Error Rate: 0.0367
Epoch 90/100, Error Rate: 0.0316
Epoch 100/100, Error Rate: 0.0234

```



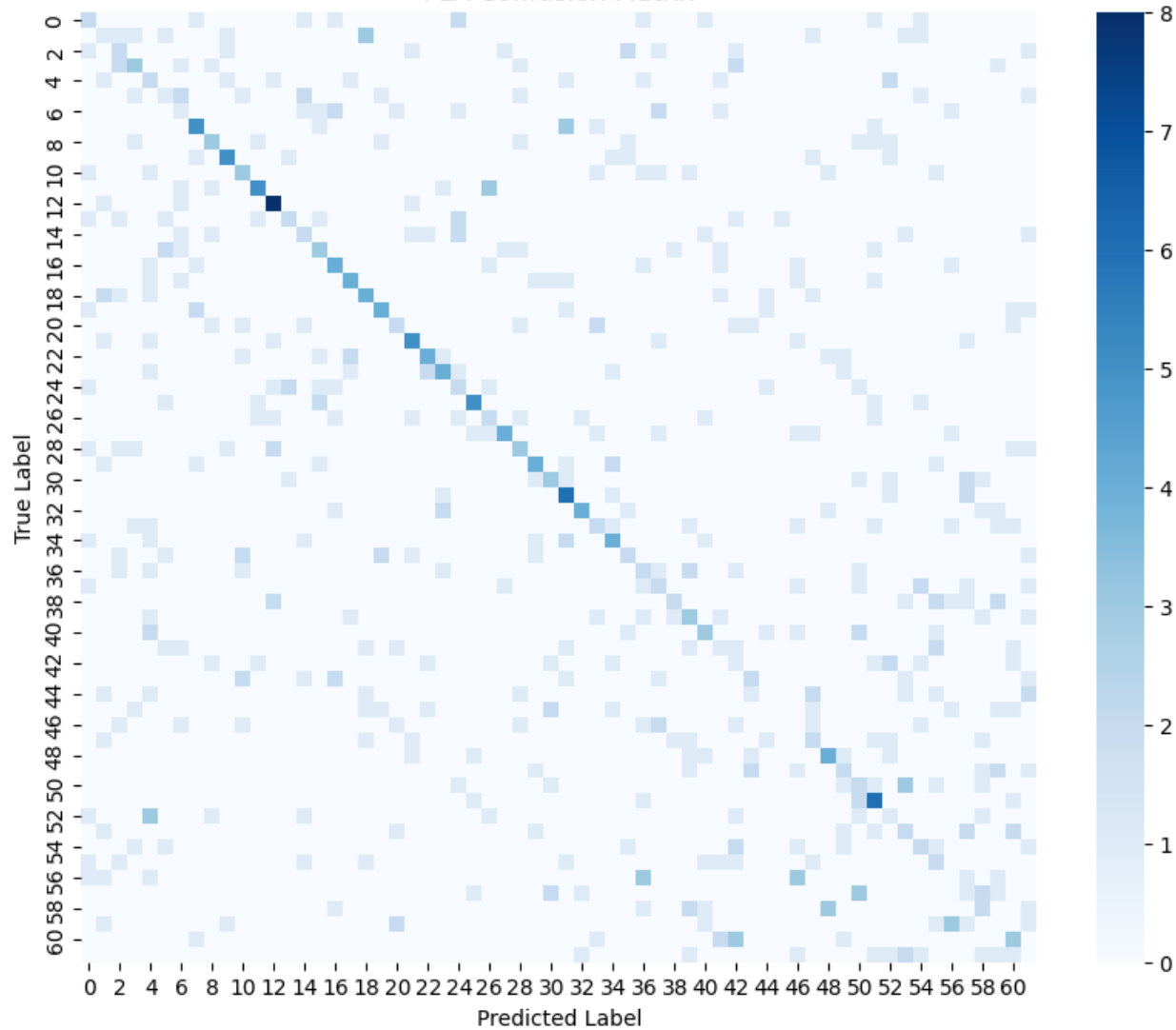
PLA Test Accuracy: 0.2419

PLA Test Precision: 0.2491

PLA Test Recall: 0.2419

PLA Test F1-Score: 0.2410

PLA Confusion Matrix



Classification Report for PLA:

precision recall f1-score support

0	0.15	0.18	0.17	11
1	0.08	0.09	0.09	11
2	0.17	0.18	0.17	11
3	0.30	0.27	0.29	11
4	0.11	0.18	0.13	11
5	0.11	0.09	0.10	11
6	0.09	0.09	0.09	11
7	0.42	0.45	0.43	11
8	0.33	0.27	0.30	11
9	0.50	0.45	0.48	11
A	0.25	0.27	0.26	11
B	0.50	0.45	0.48	11
C	0.50	0.73	0.59	11
D	0.33	0.18	0.24	11
E	0.18	0.18	0.18	11
F	0.27	0.27	0.27	11
G	0.31	0.36	0.33	11
H	0.44	0.36	0.40	11
I	0.33	0.36	0.35	11
J	0.44	0.36	0.40	11
K	0.25	0.18	0.21	11
L	0.42	0.45	0.43	11
M	0.44	0.36	0.40	11
N	0.40	0.36	0.38	11
O	0.17	0.18	0.17	11
P	0.56	0.45	0.50	11
Q	0.22	0.18	0.20	11
R	0.57	0.36	0.44	11
S	0.27	0.27	0.27	11
T	0.40	0.36	0.38	11
U	0.30	0.27	0.29	11
V	0.32	0.55	0.40	11
W	0.57	0.36	0.44	11
X	0.22	0.18	0.20	11
Y	0.33	0.36	0.35	11
Z	0.22	0.18	0.20	11
a	0.14	0.18	0.16	11
b	0.17	0.18	0.17	11
c	0.40	0.18	0.25	11
d	0.21	0.27	0.24	11
e	0.27	0.27	0.27	11
f	0.10	0.09	0.10	11
g	0.06	0.09	0.07	11
h	0.29	0.18	0.22	11
i	0.00	0.00	0.00	11
j	0.00	0.00	0.00	11
k	0.00	0.00	0.00	11
l	0.18	0.18	0.18	11
m	0.40	0.36	0.38	11
n	0.22	0.18	0.20	11
o	0.13	0.18	0.15	11

```

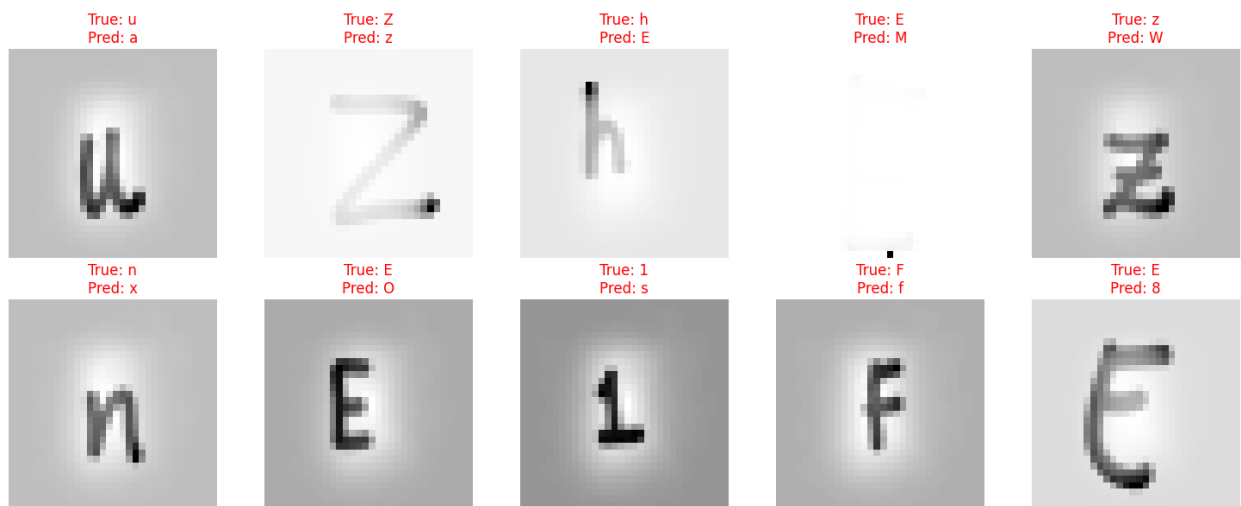
p 0.32 0.55 0.40 11
q 0.08 0.09 0.09 11
r 0.17 0.18 0.17 11
s 0.18 0.18 0.18 11
t 0.15 0.18 0.17 11
u 0.00 0.00 0.00 11
v 0.08 0.09 0.09 11
w 0.17 0.18 0.17 11
x 0.00 0.00 0.00 11
y 0.21 0.27 0.24 11
z 0.00 0.00 0.00 11

```

```

accuracy 0.24 682
macro avg 0.25 0.24 0.24 682
weighted avg 0.25 0.24 0.24 682

```



3.MLP Implementation and Results

In[7]:

```

# Define MLP model
class MLP(nn.Module):
    def __init__(self, input_size, hidden_sizes, output_size, activation='relu'):
        super(MLP, self).__init__()

        # Create layers
        layers=[]
        prev_size=input_size

        for hidden_size in hidden_sizes:
            layers.append(nn.Linear(prev_size, hidden_size))

            # Add activation function
            if activation=='relu':
                layers.append(nn.ReLU())
            elif activation=='sigmoid':
                layers.append(nn.Sigmoid())

```

```

        elif activation == 'tanh':
            layers.append(nn.Tanh())

        # Add dropout for regularization
        layers.append(nn.Dropout(dropout_rate))

        prev_size = hidden_size

    # Output layer
    layers.append(nn.Linear(prev_size, output_size))

    self.model = nn.Sequential(*layers)

    def forward(self, x):
        return self.model(x)

# Training function with detailed metrics tracking
def train_model(model, train_loader, val_loader, criterion, optimizer, num_epochs):
    train_losses = []
    val_losses = []
    train_accuracies = []
    val_accuracies = []

    best_val_acc = 0.0
    epochs_no_improve = 0
    best_model_state = None

    for epoch in range(num_epochs):
        # Training phase
        model.train()
        running_loss = 0.0
        train_correct = 0
        train_total = 0

        for inputs, labels in train_loader:
            optimizer.zero_grad()
            outputs = model(inputs)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()

            running_loss += loss.item() * inputs.size(0)

        # Calculate training accuracy
        _, predicted = torch.max(outputs.data, 1)
        train_total += labels.size(0)
        train_correct += (predicted == labels).sum().item()

        epoch_train_loss = running_loss / len(train_loader.dataset)
        epoch_train_acc = train_correct / train_total
        train_losses.append(epoch_train_loss)
        train_accuracies.append(epoch_train_acc)

```

```

# Validation phase
model.eval()
val_loss=0.0
val_correct=0
val_total=0

with torch.no_grad():
    for inputs, labels in val_loader:
        outputs = model(inputs)
        loss = criterion(outputs, labels)
        val_loss += loss.item() * inputs.size(0)

        _, predicted = torch.max(outputs.data, 1)
        val_total += labels.size(0)
        val_correct += (predicted == labels).sum().item()

epoch_val_loss = val_loss / len(val_loader.dataset)
epoch_val_acc = val_correct / val_total
val_losses.append(epoch_val_loss)
val_accuracies.append(epoch_val_acc)

# Check for improvement
if epoch_val_acc > best_val_acc:
    best_val_acc = epoch_val_acc
    best_model_state = model.state_dict().copy()
    epochs_no_improve = 0
else:
    epochs_no_improve += 1

# Early stopping
if epochs_no_improve >= patience:
    print(f"Early stopping at epoch {epoch+1}")
    break

if (epoch+1) % 10 == 0:
    print(f"Epoch {epoch+1} / {num_epochs}, Train Loss: {epoch_train_loss}")
    print(f"Train Acc: {epoch_train_acc:.4f}, Val Acc: {epoch_val_acc:.4f}")

# Load best model
if best_model_state is not None:
    model.load_state_dict(best_model_state)

return model, train_losses, val_losses, train_accuracies, val_accuracies

# Evaluation function with probability outputs
def evaluate_model(model, test_loader):
    model.eval()
    correct = 0
    total = 0
    all_preds = []
    all_labels = []
    all_probs = []

```

```

with torch.no_grad():
    for inputs, labels in test_loader:
        outputs = model(inputs)
        probs = torch.softmax(outputs, dim=1)
        _, predicted = torch.max(outputs.data, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

    all_preds.extend(predicted.cpu().numpy())
    all_labels.extend(labels.cpu().numpy())
    all_probs.extend(probs.cpu().numpy())

accuracy = correct / total
return accuracy, np.array(all_preds), np.array(all_labels), np.array(all_probs)

# — IMPORTANT: ensure DataLoaders exist (this is the minimal required fix) —
# You already created train_dataset, val_dataset, test_dataset earlier.
# Create matching DataLoader objects expected by the training/eval code:

batch_size = 64 # adjust if you want
train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)
test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)

# Hyperparameter tuning
hidden_layers_options = [[128], [256], [128, 64], [256, 128]]
activation_options = ['relu', 'sigmoid', 'tanh']
learning_rates = [0.001, 0.01, 0.1]
optimizers_list = ['adam', 'sgd']

# Tuning loop
best_val_acc = 0
best_params = {}
best_model = None
all_results = []

print("Starting hyperparameter tuning...")

input_size = X_train.shape[1]
num_classes = len(np.unique(y_train))

for hidden_layers in hidden_layers_options:
    for activation in activation_options:
        for lr in learning_rates:
            for opt_name in optimizers_list:
                print(f"Testing: Layers={hidden_layers}, Activation={activation}, "
                      f"Learning Rate={lr}, Optimizer={opt_name}")

                # Initialize model
                model = MLP(input_size, hidden_layers, num_classes, activation=activation,
                             learning_rate=lr, optimizer=opt_name)

                # Define loss and optimizer

```

```

if opt_name == 'adam':
    optimizer = optim.Adam(model.parameters(), lr=lr)
else:
    optimizer = optim.SGD(model.parameters(), lr=lr, momentum=0.9

# Train model
model, train_losses, val_losses, train_accs, val_accs = train_model(
    model, train_loader, val_loader, criterion, optimizer,
    num_epochs=50, patience=7
)

# Evaluate on validation set
val_acc, _, _, _ = evaluate_model(model, val_loader)

print(f"Validation Accuracy: {val_acc:.4f}")

# Store results
result = {
    'hidden_layers': hidden_layers,
    'activation': activation,
    'learning_rate': lr,
    'optimizer': opt_name,
    'val_accuracy': val_acc,
    'train_losses': train_losses,
    'val_losses': val_losses,
    'train_accs': train_accs,
    'val_accs': val_accs
}
all_results.append(result)

# Update best model
if val_acc > best_val_acc:
    best_val_acc = val_acc
    best_params = {
        'hidden_layers': hidden_layers,
        'activation': activation,
        'learning_rate': lr,
        'optimizer': opt_name
    }
    best_model = model
    print("New best model found!")

print("Tuning completed!")
print(f"Best validation accuracy: {best_val_acc:.4f}")
print(f"Best parameters: {best_params}")

# Plot training curves for all configurations
fig, axes = plt.subplots(2, 2, figsize=(15, 12))
axes = axes.flatten()

for i, result in enumerate(all_results):
    axes[0].plot(result['train_losses'], label=f"Config {i+1}")
    axes[1].plot(result['val_losses'], label=f"Config {i+1}")

```

```

        axes[2].plot(result['train_accs'],label=f"Config{i+1}")
        axes[3].plot(result['val_accs'],label=f"Config{i+1}")

axes[0].set_title('Training Loss')
axes[0].set_xlabel('Epoch')
axes[0].set_ylabel('Loss')
axes[0].legend()

axes[1].set_title('Validation Loss')
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Loss')
axes[1].legend()

axes[2].set_title('Training Accuracy')
axes[2].set_xlabel('Epoch')
axes[2].set_ylabel('Accuracy')
axes[2].legend()

axes[3].set_title('Validation Accuracy')
axes[3].set_xlabel('Epoch')
axes[3].set_ylabel('Accuracy')
axes[3].legend()

plt.tight_layout()
plt.show()

# Train the best model with more epochs
print("Training best model with optimal parameters...")

# Reinitialize best model with best parameters
final_model=MLP(
    input_size,
    best_params['hidden_layers'],
    num_classes,
    activation=best_params['activation']
)

criterion=nn.CrossEntropyLoss()

ifbest_params['optimizer']=='adam':
    optimizer=optim.Adam(final_model.parameters(),lr=best_params['learning_rate']
else:
    optimizer=optim.SGD(final_model.parameters(),lr=best_params['learning_rate']

# Train the model
final_model,train_losses,val_losses,train_accs,val_accs=train_model(
    final_model,train_loader,val_loader,criterion,optimizer,
    num_epochs=100,patience=10
)

# Plot training history
plt.figure(figsize=(15,5))

```

```

plt.subplot(1,3,1)
plt.plot(train_losses,label='Training Loss')
plt.plot(val_losses,label='Validation Loss')
plt.title('Training and Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()

plt.subplot(1,3,2)
plt.plot(train_accs,label='Training Accuracy')
plt.plot(val_accs,label='Validation Accuracy')
plt.title('Training and Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()

# Plot learning curve (accuracy vs loss)
plt.subplot(1,3,3)
plt.plot(train_losses,train_accs,'o-',label='Training')
plt.plot(val_losses,val_accs,'o-',label='Validation')
plt.title('Learning Curve')
plt.xlabel('Loss')
plt.ylabel('Accuracy')
plt.legend()

plt.tight_layout()
plt.show()

# Evaluate on test set
mlp_test_acc,mlp_preds,mlp_true,mlp_probs=evaluate_model(final_model,test_loader)
mlp_precision=precision_score(mlp_true,mlp_preds,average='weighted',zero_division
mlp_recall=recall_score(mlp_true,mlp_preds,average='weighted',zero_division
mlp_f1=f1_score(mlp_true,mlp_preds,average='weighted',zero_division=0)

print(f"MLP Test Accuracy:{mlp_test_acc:.4f}")
print(f"MLP Test Precision:{mlp_precision:.4f}")
print(f"MLP Test Recall:{mlp_recall:.4f}")
print(f"MLP Test F1-Score:{mlp_f1:.4f}")

# Confusion matrix for MLP
plt.figure(figsize=(10,8))
cm_mlp=confusion_matrix(mlp_true,mlp_preds)
sns.heatmap(cm_mlp,annot=False,fmt='d',cmap='Blues')
plt.title('MLP Confusion Matrix')
plt.ylabel('True Label')
plt.xlabel('Predicted Label')
plt.show()

# ROC Curves
# Binarize the output for ROC curve
y_test_bin_mlp=label_binarize(mlp_true,classes=np.arange(num_classes))

# Compute ROC curve and ROC area for each class for MLP

```



```

fpr_mlp=dict()
tpr_mlp=dict()
roc_auc_mlp=dict()

foriinrange(num_classes):
    fpr_mlp[i],tpr_mlp[i],_=roc_curve(y_test_bin_mlp[:,i],mlp_probs[:,i])
    roc_auc_mlp[i]=auc(fpr_mlp[i],tpr_mlp[i])

# Compute micro-average ROC curve and ROC area for MLP
fpr_mlp["micro"],tpr_mlp["micro"],_=roc_curve(y_test_bin_mlp.ravel(),mlp_probs)
roc_auc_mlp["micro"]=auc(fpr_mlp["micro"],tpr_mlp["micro"])

# Plot ROC curves
plt.figure(figsize=(12,5))

plt.subplot(1,2,1)
plt.plot(fpr_mlp["micro"],tpr_mlp["micro"],
         label='MLP micro-average ROC curve (area ={0:0.2f})'
         ".format(roc_auc_mlp["micro"]),
         color='navy',linestyle=':',linewidth=4)

plt.plot([0,1],[0,1],'k-',lw=2)
plt.xlim([0.0,1.0])
plt.ylim([0.0,1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Micro-average ROC Curve')
plt.legend(loc="lower right")

# Macro-average ROC curve
all_fpr_mlp=np.unique(np.concatenate([fpr_mlp[i]foriinrange(num_classes)]))

mean_tpr_mlp=np.zeros_like(all_fpr_mlp)
foriinrange(num_classes):
    mean_tpr_mlp+=np.interp(all_fpr_mlp,fpr_mlp[i],tpr_mlp[i])
mean_tpr_mlp/=num_classes

fpr_mlp["macro"]=all_fpr_mlp
tpr_mlp["macro"]=mean_tpr_mlp
roc_auc_mlp["macro"]=auc(fpr_mlp["macro"],tpr_mlp["macro"])

plt.subplot(1,2,2)
plt.plot(fpr_mlp["macro"],tpr_mlp["macro"],
         label='MLP macro-average ROC curve (area ={0:0.2f})'
         ".format(roc_auc_mlp["macro"]),
         color='navy',linestyle=':',linewidth=4)

plt.plot([0,1],[0,1],'k-',lw=2)
plt.xlim([0.0,1.0])
plt.ylim([0.0,1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Macro-average ROC Curve')

```

```

plt.legend(loc="lower right")

plt.tight_layout()
plt.show()

# Classification report for MLP
class_names=label_encoder.classes_
print("Report for MLP:")
print(classification_report(mlp_true,mlp_preds,target_names=class_names,zero_division

# Visualize some predictions
defvisualize_mlp_predictions(model,test_loader,label_encoder,num_samples=10
    model.eval()
    images,labels=next(iter(test_loader))

    withtorch.no_grad():
        outputs=model(images)
        _,preds=torch.max(outputs,1)

    plt.figure(figsize=(15,6))
    foriinrange(num_samples):
        plt.subplot(2,5,i+1)
        # Reshape flattened image to 2D (assuming original was 32x32)
        img=images[i].numpy().reshape(32,32)
        plt.imshow(img,cmap='gray')

        true_label=label_encoder.inverse_transform([labels[i].item()])[0]
        pred_label=label_encoder.inverse_transform([preds[i].item()])[0]

        title_color='green'iftrue_label==pred_labelelse'red'
        plt.title(f"True:{true_label}:{pred_label}",color=title_color)
        plt.axis('off')

    plt.tight_layout()
    plt.show()

visualize_mlp_predictions(final_model,test_loader,label_encoder)

```

Starting hyperparameter tuning...

Testing: Layers=[128], Activation=relu, LR=0.001, Optimizer=adam
Epoch 10/50, Train Loss: 1.3544, Val Loss: 2.5509
Train Acc: 0.6508, Val Acc: 0.4176
Epoch 20/50, Train Loss: 0.7898, Val Loss: 2.7088
Train Acc: 0.7947, Val Acc: 0.4432
Early stopping at epoch 27
Validation Accuracy: 0.4341
New best model found!

Testing: Layers=[128], Activation=relu, LR=0.001, Optimizer=sgd
Epoch 10/50, Train Loss: 3.8226, Val Loss: 3.9923
Train Acc: 0.1815, Val Acc: 0.1190
Epoch 20/50, Train Loss: 3.4158, Val Loss: 3.7265
Train Acc: 0.2988, Val Acc: 0.2125
Epoch 30/50, Train Loss: 2.9644, Val Loss: 3.4132
Train Acc: 0.3648, Val Acc: 0.2674
Epoch 40/50, Train Loss: 2.5825, Val Loss: 3.1462
Train Acc: 0.4244, Val Acc: 0.2857
Epoch 50/50, Train Loss: 2.2843, Val Loss: 2.9503
Train Acc: 0.4853, Val Acc: 0.3168
Validation Accuracy: 0.3168

Testing: Layers=[128], Activation=relu, LR=0.01, Optimizer=adam
Epoch 10/50, Train Loss: 1.7571, Val Loss: 16.2614
Train Acc: 0.6522, Val Acc: 0.3535
Early stopping at epoch 18
Validation Accuracy: 0.3608

Testing: Layers=[128], Activation=relu, LR=0.01, Optimizer=sgd
Epoch 10/50, Train Loss: 1.6727, Val Loss: 2.5893
Train Acc: 0.5770, Val Acc: 0.3864
Epoch 20/50, Train Loss: 1.0510, Val Loss: 2.5537
Train Acc: 0.7172, Val Acc: 0.4139
Epoch 30/50, Train Loss: 0.7251, Val Loss: 2.6585
Train Acc: 0.8038, Val Acc: 0.4524
Epoch 40/50, Train Loss: 0.5410, Val Loss: 2.8144
Train Acc: 0.8625, Val Acc: 0.4524
Early stopping at epoch 49
Validation Accuracy: 0.4762
New best model found!

Testing: Layers=[128], Activation=relu, LR=0.1, Optimizer=adam
Epoch 10/50, Train Loss: 119.7129, Val Loss: 1336.2006
Train Acc: 0.3721, Val Acc: 0.2125
Epoch 20/50, Train Loss: 78.8579, Val Loss: 2160.3125
Train Acc: 0.5160, Val Acc: 0.2308
Early stopping at epoch 29
Validation Accuracy: 0.2344

Testing: Layers=[128], Activation=relu, LR=0.1, Optimizer=sgd
Early stopping at epoch 9
Validation Accuracy: 0.1081

Testing: Layers=[128], Activation=sigmoid, LR=0.001, Optimizer=adam
Epoch 10/50, Train Loss: 2.5759, Val Loss: 2.9362
Train Acc: 0.4661, Val Acc: 0.3388
Epoch 20/50, Train Loss: 1.8498, Val Loss: 2.5791
Train Acc: 0.5921, Val Acc: 0.3938
Epoch 30/50, Train Loss: 1.4968, Val Loss: 2.4843

Train Acc: 0.6645, Val Acc: 0.4029
Epoch 40/50, Train Loss: 1.2266, Val Loss: 2.4717
Train Acc: 0.7218, Val Acc: 0.4011
Early stopping at epoch 42
Validation Accuracy: 0.3993
Testing: Layers=[128], Activation=sigmoid, LR=0.001, Optimizer=sgd
Epoch 10/50, Train Loss: 4.0917, Val Loss: 4.0930
Train Acc: 0.0289, Val Acc: 0.0531
Epoch 20/50, Train Loss: 4.0312, Val Loss: 4.0486
Train Acc: 0.0669, Val Acc: 0.1026
Epoch 30/50, Train Loss: 3.9577, Val Loss: 3.9968
Train Acc: 0.1022, Val Acc: 0.1410
Epoch 40/50, Train Loss: 3.8811, Val Loss: 3.9350
Train Acc: 0.1444, Val Acc: 0.1648
Epoch 50/50, Train Loss: 3.7872, Val Loss: 3.8644
Train Acc: 0.1829, Val Acc: 0.2015
Validation Accuracy: 0.2015
Testing: Layers=[128], Activation=sigmoid, LR=0.01, Optimizer=adam
Epoch 10/50, Train Loss: 0.9654, Val Loss: 2.5576
Train Acc: 0.7456, Val Acc: 0.4029
Epoch 20/50, Train Loss: 0.4501, Val Loss: 2.8729
Train Acc: 0.8905, Val Acc: 0.4011
Early stopping at epoch 21
Validation Accuracy: 0.4139
Testing: Layers=[128], Activation=sigmoid, LR=0.01, Optimizer=sgd
Epoch 10/50, Train Loss: 3.4204, Val Loss: 3.5206
Train Acc: 0.2397, Val Acc: 0.2216
Epoch 20/50, Train Loss: 2.6881, Val Loss: 2.9726
Train Acc: 0.3955, Val Acc: 0.3315
Epoch 30/50, Train Loss: 2.2594, Val Loss: 2.6994
Train Acc: 0.4826, Val Acc: 0.3681
Epoch 40/50, Train Loss: 1.9753, Val Loss: 2.5715
Train Acc: 0.5275, Val Acc: 0.3864
Epoch 50/50, Train Loss: 1.8011, Val Loss: 2.5047
Train Acc: 0.5541, Val Acc: 0.3938
Validation Accuracy: 0.3938
Testing: Layers=[128], Activation=sigmoid, LR=0.1, Optimizer=adam
Epoch 10/50, Train Loss: 3.7486, Val Loss: 5.0157
Train Acc: 0.3208, Val Acc: 0.2601
Early stopping at epoch 20
Validation Accuracy: 0.2289
Testing: Layers=[128], Activation=sigmoid, LR=0.1, Optimizer=sgd
Epoch 10/50, Train Loss: 1.5182, Val Loss: 2.5779
Train Acc: 0.5972, Val Acc: 0.3810
Epoch 20/50, Train Loss: 0.8584, Val Loss: 2.6220
Train Acc: 0.7764, Val Acc: 0.3993
Early stopping at epoch 23
Validation Accuracy: 0.4084
Testing: Layers=[128], Activation=tanh, LR=0.001, Optimizer=adam
Epoch 10/50, Train Loss: 1.5962, Val Loss: 2.5770
Train Acc: 0.6379, Val Acc: 0.3755
Epoch 20/50, Train Loss: 1.0176, Val Loss: 2.5404
Train Acc: 0.7704, Val Acc: 0.4011
Epoch 30/50, Train Loss: 0.7035, Val Loss: 2.6326

Train Acc: 0.8488, Val Acc: 0.3919
Early stopping at epoch 31
Validation Accuracy: 0.3993
Testing: Layers=[128], Activation=tanh, LR=0.001, Optimizer=sgd
Epoch 10/50, Train Loss: 3.5857, Val Loss: 3.7337
Train Acc: 0.2383, Val Acc: 0.1685
Epoch 20/50, Train Loss: 3.0999, Val Loss: 3.3904
Train Acc: 0.3689, Val Acc: 0.2399
Epoch 30/50, Train Loss: 2.7370, Val Loss: 3.1432
Train Acc: 0.4345, Val Acc: 0.2894
Epoch 40/50, Train Loss: 2.4745, Val Loss: 2.9725
Train Acc: 0.4821, Val Acc: 0.3187
Epoch 50/50, Train Loss: 2.2703, Val Loss: 2.8500
Train Acc: 0.5257, Val Acc: 0.3462
Validation Accuracy: 0.3462
Testing: Layers=[128], Activation=tanh, LR=0.01, Optimizer=adam
Epoch 10/50, Train Loss: 0.7342, Val Loss: 2.9591
Train Acc: 0.7919, Val Acc: 0.3791
Early stopping at epoch 14
Validation Accuracy: 0.4011
Testing: Layers=[128], Activation=tanh, LR=0.01, Optimizer=sgd
Epoch 10/50, Train Loss: 1.8323, Val Loss: 2.6104
Train Acc: 0.5926, Val Acc: 0.3901
Epoch 20/50, Train Loss: 1.2537, Val Loss: 2.5422
Train Acc: 0.7159, Val Acc: 0.3938
Epoch 30/50, Train Loss: 0.9171, Val Loss: 2.5748
Train Acc: 0.7929, Val Acc: 0.4103
Early stopping at epoch 39
Validation Accuracy: 0.4029
Testing: Layers=[128], Activation=tanh, LR=0.1, Optimizer=adam
Epoch 10/50, Train Loss: 7.6051, Val Loss: 8.9833
Train Acc: 0.2741, Val Acc: 0.2106
Epoch 20/50, Train Loss: 7.3766, Val Loss: 10.1003
Train Acc: 0.3181, Val Acc: 0.2436
Early stopping at epoch 25
Validation Accuracy: 0.2399
Testing: Layers=[128], Activation=tanh, LR=0.1, Optimizer=sgd
Epoch 10/50, Train Loss: 0.7780, Val Loss: 3.0385
Train Acc: 0.7777, Val Acc: 0.4011
Early stopping at epoch 17
Validation Accuracy: 0.4011
Testing: Layers=[256], Activation=relu, LR=0.001, Optimizer=adam
Epoch 10/50, Train Loss: 0.9582, Val Loss: 2.5474
Train Acc: 0.7612, Val Acc: 0.4341
Early stopping at epoch 16
Validation Accuracy: 0.4432
Testing: Layers=[256], Activation=relu, LR=0.001, Optimizer=sgd
Epoch 10/50, Train Loss: 3.7814, Val Loss: 3.9877
Train Acc: 0.2250, Val Acc: 0.1740
Epoch 20/50, Train Loss: 3.3471, Val Loss: 3.7294
Train Acc: 0.3318, Val Acc: 0.2399
Epoch 30/50, Train Loss: 2.8768, Val Loss: 3.4159
Train Acc: 0.3996, Val Acc: 0.2747
Epoch 40/50, Train Loss: 2.4703, Val Loss: 3.1467

Train Acc: 0.4803, Val Acc: 0.3040
Epoch 50/50, Train Loss: 2.1618, Val Loss: 2.9501
Train Acc: 0.5257, Val Acc: 0.3297
Validation Accuracy: 0.3297
Testing: Layers=[256], Activation=relu, LR=0.01, Optimizer=adam
Epoch 10/50, Train Loss: 1.9447, Val Loss: 24.1263
Train Acc: 0.6806, Val Acc: 0.3443
Epoch 20/50, Train Loss: 1.1789, Val Loss: 36.7466
Train Acc: 0.8387, Val Acc: 0.3590
Early stopping at epoch 26
Validation Accuracy: 0.3901
Testing: Layers=[256], Activation=relu, LR=0.01, Optimizer=sgd
Epoch 10/50, Train Loss: 1.5194, Val Loss: 2.5457
Train Acc: 0.6269, Val Acc: 0.4194
Epoch 20/50, Train Loss: 0.8375, Val Loss: 2.5639
Train Acc: 0.7846, Val Acc: 0.4396
Epoch 30/50, Train Loss: 0.5045, Val Loss: 2.7001
Train Acc: 0.8831, Val Acc: 0.4469
Early stopping at epoch 35
Validation Accuracy: 0.4560
Testing: Layers=[256], Activation=relu, LR=0.1, Optimizer=adam
Epoch 10/50, Train Loss: 220.4494, Val Loss: 2010.2591
Train Acc: 0.5128, Val Acc: 0.2912
Early stopping at epoch 20
Validation Accuracy: 0.3150
Testing: Layers=[256], Activation=relu, LR=0.1, Optimizer=sgd
Early stopping at epoch 9
Validation Accuracy: 0.0971
Testing: Layers=[256], Activation=sigmoid, LR=0.001, Optimizer=adam
Epoch 10/50, Train Loss: 2.1530, Val Loss: 2.6899
Train Acc: 0.5339, Val Acc: 0.3535
Epoch 20/50, Train Loss: 1.4811, Val Loss: 2.4742
Train Acc: 0.6641, Val Acc: 0.4121
Epoch 30/50, Train Loss: 1.1213, Val Loss: 2.4429
Train Acc: 0.7356, Val Acc: 0.4286
Early stopping at epoch 38
Validation Accuracy: 0.4267
Testing: Layers=[256], Activation=sigmoid, LR=0.001, Optimizer=sgd
Epoch 10/50, Train Loss: 4.0883, Val Loss: 4.0881
Train Acc: 0.0330, Val Acc: 0.0604
Epoch 20/50, Train Loss: 4.0166, Val Loss: 4.0411
Train Acc: 0.0724, Val Acc: 0.1209
Epoch 30/50, Train Loss: 3.9454, Val Loss: 3.9834
Train Acc: 0.1159, Val Acc: 0.1337
Early stopping at epoch 31
Validation Accuracy: 0.1484
Testing: Layers=[256], Activation=sigmoid, LR=0.01, Optimizer=adam
Epoch 10/50, Train Loss: 0.5480, Val Loss: 2.6941
Train Acc: 0.8749, Val Acc: 0.4066
Epoch 20/50, Train Loss: 0.2043, Val Loss: 3.0546
Train Acc: 0.9565, Val Acc: 0.4066
Early stopping at epoch 26
Validation Accuracy: 0.4194
Testing: Layers=[256], Activation=sigmoid, LR=0.01, Optimizer=sgd

Epoch 10/50, Train Loss: 3.2844, Val Loss: 3.3970
Train Acc: 0.2681, Val Acc: 0.2619
Epoch 20/50, Train Loss: 2.4770, Val Loss: 2.8327
Train Acc: 0.4230, Val Acc: 0.3535
Epoch 30/50, Train Loss: 2.0761, Val Loss: 2.6133
Train Acc: 0.5060, Val Acc: 0.3700
Epoch 40/50, Train Loss: 1.8355, Val Loss: 2.5268
Train Acc: 0.5545, Val Acc: 0.4011
Early stopping at epoch 50
Validation Accuracy: 0.4103
Testing: Layers=[256], Activation=sigmoid, LR=0.1, Optimizer=adam
Epoch 10/50, Train Loss: 5.5293, Val Loss: 8.0138
Train Acc: 0.3226, Val Acc: 0.2271
Epoch 20/50, Train Loss: 7.0722, Val Loss: 9.2096
Train Acc: 0.3905, Val Acc: 0.2692
Epoch 30/50, Train Loss: 4.5695, Val Loss: 9.8246
Train Acc: 0.4991, Val Acc: 0.2637
Early stopping at epoch 32
Validation Accuracy: 0.2454
Testing: Layers=[256], Activation=sigmoid, LR=0.1, Optimizer=sgd
Epoch 10/50, Train Loss: 1.4362, Val Loss: 2.7509
Train Acc: 0.6127, Val Acc: 0.3681
Epoch 20/50, Train Loss: 0.6980, Val Loss: 2.8485
Train Acc: 0.8093, Val Acc: 0.4176
Early stopping at epoch 25
Validation Accuracy: 0.4139
Testing: Layers=[256], Activation=tanh, LR=0.001, Optimizer=adam
Epoch 10/50, Train Loss: 1.2662, Val Loss: 2.5986
Train Acc: 0.7030, Val Acc: 0.3993
Early stopping at epoch 19
Validation Accuracy: 0.3919
Testing: Layers=[256], Activation=tanh, LR=0.001, Optimizer=sgd
Epoch 10/50, Train Loss: 3.5208, Val Loss: 3.6808
Train Acc: 0.2786, Val Acc: 0.2033
Epoch 20/50, Train Loss: 2.9368, Val Loss: 3.2739
Train Acc: 0.4038, Val Acc: 0.2766
Epoch 30/50, Train Loss: 2.5355, Val Loss: 3.0110
Train Acc: 0.4661, Val Acc: 0.3168
Epoch 40/50, Train Loss: 2.2591, Val Loss: 2.8459
Train Acc: 0.5211, Val Acc: 0.3388
Epoch 50/50, Train Loss: 2.0609, Val Loss: 2.7408
Train Acc: 0.5674, Val Acc: 0.3663
Validation Accuracy: 0.3663
Testing: Layers=[256], Activation=tanh, LR=0.01, Optimizer=adam
Epoch 10/50, Train Loss: 0.4670, Val Loss: 3.3897
Train Acc: 0.8584, Val Acc: 0.4048
Early stopping at epoch 17
Validation Accuracy: 0.3755
Testing: Layers=[256], Activation=tanh, LR=0.01, Optimizer=sgd
Epoch 10/50, Train Loss: 1.6578, Val Loss: 2.5736
Train Acc: 0.6219, Val Acc: 0.3938
Epoch 20/50, Train Loss: 1.1107, Val Loss: 2.6197
Train Acc: 0.7411, Val Acc: 0.3956
Early stopping at epoch 23

Validation Accuracy: 0.3974
Testing: Layers=[256], Activation=tanh, LR=0.1, Optimizer=adam
Epoch 10/50, Train Loss: 12.2851, Val Loss: 16.5324
Train Acc: 0.3286, Val Acc: 0.2546
Early stopping at epoch 16
Validation Accuracy: 0.2582
Testing: Layers=[256], Activation=tanh, LR=0.1, Optimizer=sgd
Epoch 10/50, Train Loss: 0.4606, Val Loss: 3.5770
Train Acc: 0.8602, Val Acc: 0.4048
Epoch 20/50, Train Loss: 0.1923, Val Loss: 4.2308
Train Acc: 0.9441, Val Acc: 0.3919
Early stopping at epoch 24
Validation Accuracy: 0.4121
Testing: Layers=[128, 64], Activation=relu, LR=0.001, Optimizer=adam
Epoch 10/50, Train Loss: 1.7749, Val Loss: 2.5438
Train Acc: 0.5101, Val Acc: 0.4048
Epoch 20/50, Train Loss: 1.2077, Val Loss: 2.7370
Train Acc: 0.6398, Val Acc: 0.4487
Early stopping at epoch 29
Validation Accuracy: 0.4597
Testing: Layers=[128, 64], Activation=relu, LR=0.001, Optimizer=sgd
Epoch 10/50, Train Loss: 4.0869, Val Loss: 4.1570
Train Acc: 0.0408, Val Acc: 0.0385
Epoch 20/50, Train Loss: 4.0162, Val Loss: 4.1294
Train Acc: 0.0692, Val Acc: 0.0604
Epoch 30/50, Train Loss: 3.9038, Val Loss: 4.0658
Train Acc: 0.0885, Val Acc: 0.0879
Epoch 40/50, Train Loss: 3.7102, Val Loss: 3.9335
Train Acc: 0.1407, Val Acc: 0.1374
Epoch 50/50, Train Loss: 3.4562, Val Loss: 3.7232
Train Acc: 0.2016, Val Acc: 0.1832
Validation Accuracy: 0.1832
Testing: Layers=[128, 64], Activation=relu, LR=0.01, Optimizer=adam
Epoch 10/50, Train Loss: 2.2323, Val Loss: 2.7080
Train Acc: 0.3877, Val Acc: 0.3187
Epoch 20/50, Train Loss: 2.2455, Val Loss: 2.7388
Train Acc: 0.3877, Val Acc: 0.3315
Early stopping at epoch 21
Validation Accuracy: 0.3297
Testing: Layers=[128, 64], Activation=relu, LR=0.01, Optimizer=sgd
Epoch 10/50, Train Loss: 2.4692, Val Loss: 2.7739
Train Acc: 0.3396, Val Acc: 0.3077
Epoch 20/50, Train Loss: 1.4629, Val Loss: 2.4503
Train Acc: 0.5820, Val Acc: 0.4286
Epoch 30/50, Train Loss: 1.1102, Val Loss: 2.5551
Train Acc: 0.6609, Val Acc: 0.4670
Early stopping at epoch 33
Validation Accuracy: 0.4670
Testing: Layers=[128, 64], Activation=relu, LR=0.1, Optimizer=adam
Early stopping at epoch 8
Validation Accuracy: 0.0147
Testing: Layers=[128, 64], Activation=relu, LR=0.1, Optimizer=sgd
Early stopping at epoch 10
Validation Accuracy: 0.0733

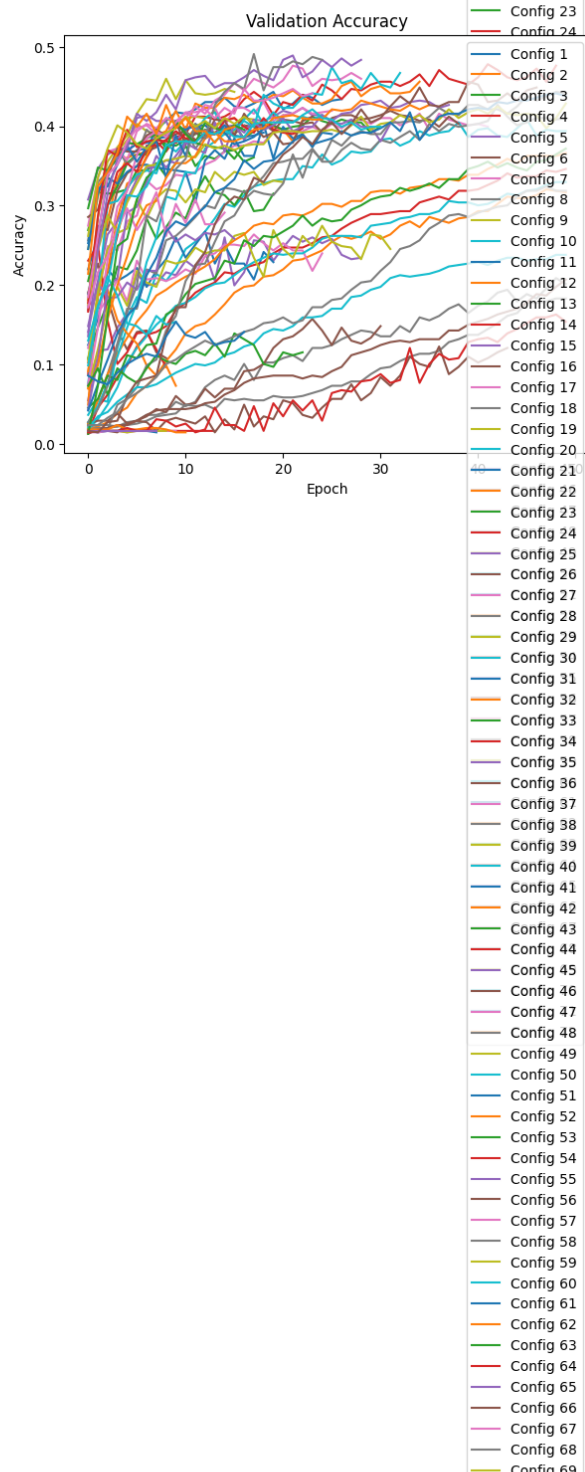
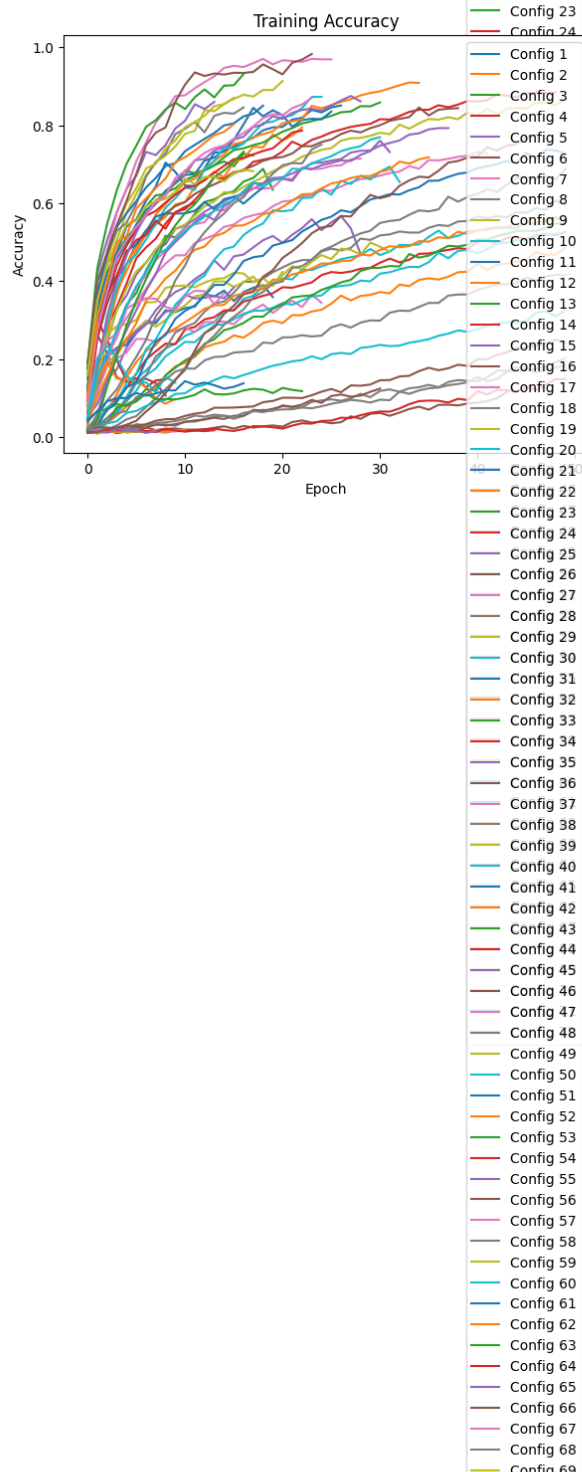
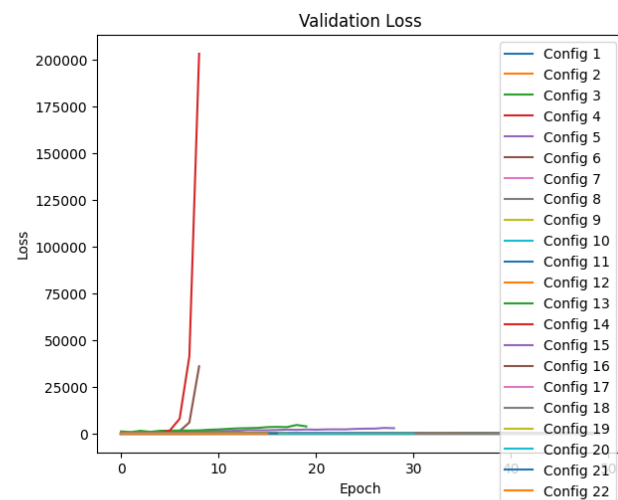
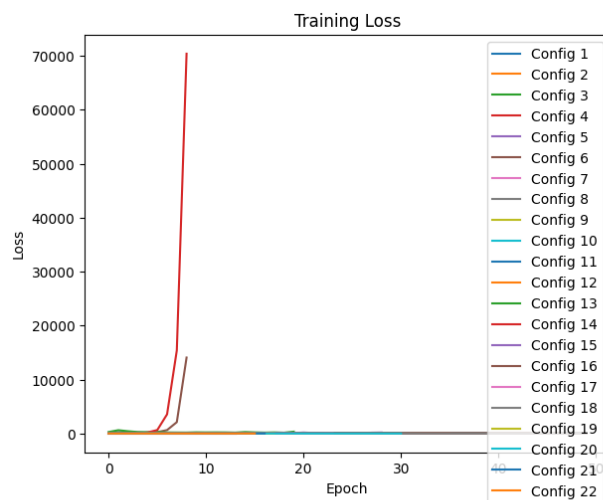
Testing: Layers=[128, 64], Activation=sigmoid, LR=0.001, Optimizer=adam
Epoch 10/50, Train Loss: 3.4206, Val Loss: 3.4650
Train Acc: 0.1778, Val Acc: 0.1703
Epoch 20/50, Train Loss: 2.7168, Val Loss: 2.9846
Train Acc: 0.3291, Val Acc: 0.2601
Epoch 30/50, Train Loss: 2.2874, Val Loss: 2.7721
Train Acc: 0.4335, Val Acc: 0.3077
Epoch 40/50, Train Loss: 1.9878, Val Loss: 2.6518
Train Acc: 0.4936, Val Acc: 0.3462
Epoch 50/50, Train Loss: 1.7757, Val Loss: 2.5771
Train Acc: 0.5477, Val Acc: 0.3718
Validation Accuracy: 0.3718
Testing: Layers=[128, 64], Activation=sigmoid, LR=0.001, Optimizer=sgd
Epoch 10/50, Train Loss: 4.1434, Val Loss: 4.1354
Train Acc: 0.0147, Val Acc: 0.0165
Early stopping at epoch 14
Validation Accuracy: 0.0165
Testing: Layers=[128, 64], Activation=sigmoid, LR=0.01, Optimizer=adam
Epoch 10/50, Train Loss: 1.7945, Val Loss: 2.5107
Train Acc: 0.5005, Val Acc: 0.3626
Epoch 20/50, Train Loss: 1.1678, Val Loss: 2.4815
Train Acc: 0.6645, Val Acc: 0.4011
Epoch 30/50, Train Loss: 0.8868, Val Loss: 2.5890
Train Acc: 0.7337, Val Acc: 0.3993
Early stopping at epoch 32
Validation Accuracy: 0.4029
Testing: Layers=[128, 64], Activation=sigmoid, LR=0.01, Optimizer=sgd
Epoch 10/50, Train Loss: 4.1333, Val Loss: 4.1199
Train Acc: 0.0220, Val Acc: 0.0330
Epoch 20/50, Train Loss: 4.0951, Val Loss: 4.0917
Train Acc: 0.0298, Val Acc: 0.0311
Epoch 30/50, Train Loss: 3.9545, Val Loss: 3.9531
Train Acc: 0.0596, Val Acc: 0.0641
Epoch 40/50, Train Loss: 3.6378, Val Loss: 3.6596
Train Acc: 0.0862, Val Acc: 0.1099
Early stopping at epoch 44
Validation Accuracy: 0.1209
Testing: Layers=[128, 64], Activation=sigmoid, LR=0.1, Optimizer=adam
Early stopping at epoch 10
Validation Accuracy: 0.0165
Testing: Layers=[128, 64], Activation=sigmoid, LR=0.1, Optimizer=sgd
Epoch 10/50, Train Loss: 3.2121, Val Loss: 3.1955
Train Acc: 0.1476, Val Acc: 0.1612
Epoch 20/50, Train Loss: 2.2261, Val Loss: 2.6408
Train Acc: 0.3937, Val Acc: 0.3132
Epoch 30/50, Train Loss: 1.6549, Val Loss: 2.4505
Train Acc: 0.5293, Val Acc: 0.3901
Epoch 40/50, Train Loss: 1.3312, Val Loss: 2.3505
Train Acc: 0.6164, Val Acc: 0.4267
Epoch 50/50, Train Loss: 1.0620, Val Loss: 2.3485
Train Acc: 0.6874, Val Acc: 0.4414
Validation Accuracy: 0.4414
Testing: Layers=[128, 64], Activation=tanh, LR=0.001, Optimizer=adam
Epoch 10/50, Train Loss: 2.0400, Val Loss: 2.6526

Train Acc: 0.5348, Val Acc: 0.3590
Epoch 20/50, Train Loss: 1.3622, Val Loss: 2.4716
Train Acc: 0.6824, Val Acc: 0.3919
Epoch 30/50, Train Loss: 0.9781, Val Loss: 2.4482
Train Acc: 0.7731, Val Acc: 0.3956
Epoch 40/50, Train Loss: 0.7317, Val Loss: 2.4597
Train Acc: 0.8332, Val Acc: 0.4176
Epoch 50/50, Train Loss: 0.5810, Val Loss: 2.5547
Train Acc: 0.8657, Val Acc: 0.4286
Validation Accuracy: 0.4286
Testing: Layers=[128, 64], Activation=tanh, LR=0.001, Optimizer=sgd
Epoch 10/50, Train Loss: 3.9226, Val Loss: 3.9612
Train Acc: 0.1104, Val Acc: 0.0897
Epoch 20/50, Train Loss: 3.6567, Val Loss: 3.7438
Train Acc: 0.1829, Val Acc: 0.1447
Epoch 30/50, Train Loss: 3.3813, Val Loss: 3.5231
Train Acc: 0.2314, Val Acc: 0.1941
Epoch 40/50, Train Loss: 3.1450, Val Loss: 3.3381
Train Acc: 0.2699, Val Acc: 0.2271
Epoch 50/50, Train Loss: 2.9374, Val Loss: 3.1902
Train Acc: 0.3245, Val Acc: 0.2381
Validation Accuracy: 0.2381
Testing: Layers=[128, 64], Activation=tanh, LR=0.01, Optimizer=adam
Epoch 10/50, Train Loss: 1.5014, Val Loss: 2.6021
Train Acc: 0.5738, Val Acc: 0.3883
Early stopping at epoch 19
Validation Accuracy: 0.3974
Testing: Layers=[128, 64], Activation=tanh, LR=0.01, Optimizer=sgd
Epoch 10/50, Train Loss: 2.4593, Val Loss: 2.7803
Train Acc: 0.4271, Val Acc: 0.3535
Epoch 20/50, Train Loss: 1.7293, Val Loss: 2.4718
Train Acc: 0.5770, Val Acc: 0.4103
Epoch 30/50, Train Loss: 1.3167, Val Loss: 2.3859
Train Acc: 0.6696, Val Acc: 0.4286
Early stopping at epoch 36
Validation Accuracy: 0.4267
Testing: Layers=[128, 64], Activation=tanh, LR=0.1, Optimizer=adam
Epoch 10/50, Train Loss: 5.4592, Val Loss: 4.9986
Train Acc: 0.0962, Val Acc: 0.1007
Epoch 20/50, Train Loss: 5.3616, Val Loss: 4.9016
Train Acc: 0.1127, Val Acc: 0.0971
Early stopping at epoch 23
Validation Accuracy: 0.1154
Testing: Layers=[128, 64], Activation=tanh, LR=0.1, Optimizer=sgd
Epoch 10/50, Train Loss: 1.4627, Val Loss: 2.6972
Train Acc: 0.5765, Val Acc: 0.3791
Early stopping at epoch 12
Validation Accuracy: 0.3791
Testing: Layers=[256, 128], Activation=relu, LR=0.001, Optimizer=adam
Epoch 10/50, Train Loss: 1.2313, Val Loss: 2.5142
Train Acc: 0.6567, Val Acc: 0.4212
Epoch 20/50, Train Loss: 0.6143, Val Loss: 2.8620
Train Acc: 0.8236, Val Acc: 0.4579
Early stopping at epoch 29

Validation Accuracy: 0.4835
New best model found!
Testing: Layers=[256, 128], Activation=relu, LR=0.001, Optimizer=sgd
Epoch 10/50, Train Loss: 4.0738, Val Loss: 4.1298
Train Acc: 0.0463, Val Acc: 0.0440
Epoch 20/50, Train Loss: 3.9861, Val Loss: 4.0806
Train Acc: 0.0926, Val Acc: 0.0861
Epoch 30/50, Train Loss: 3.8280, Val Loss: 3.9586
Train Acc: 0.1347, Val Acc: 0.1190
Epoch 40/50, Train Loss: 3.5591, Val Loss: 3.7561
Train Acc: 0.1824, Val Acc: 0.1502
Epoch 50/50, Train Loss: 3.2056, Val Loss: 3.4806
Train Acc: 0.2424, Val Acc: 0.2070
Validation Accuracy: 0.2070
Testing: Layers=[256, 128], Activation=relu, LR=0.01, Optimizer=adam
Epoch 10/50, Train Loss: 2.4591, Val Loss: 3.1112
Train Acc: 0.3263, Val Acc: 0.3022
Early stopping at epoch 15
Validation Accuracy: 0.2912
Testing: Layers=[256, 128], Activation=relu, LR=0.01, Optimizer=sgd
Epoch 10/50, Train Loss: 2.2433, Val Loss: 2.6901
Train Acc: 0.4019, Val Acc: 0.3462
Epoch 20/50, Train Loss: 1.1384, Val Loss: 2.3194
Train Acc: 0.6760, Val Acc: 0.4744
Early stopping at epoch 25
Validation Accuracy: 0.4835
Testing: Layers=[256, 128], Activation=relu, LR=0.1, Optimizer=adam
Early stopping at epoch 9
Validation Accuracy: 0.0165
Testing: Layers=[256, 128], Activation=relu, LR=0.1, Optimizer=sgd
Early stopping at epoch 10
Validation Accuracy: 0.1538
Testing: Layers=[256, 128], Activation=sigmoid, LR=0.001, Optimizer=adam
Epoch 10/50, Train Loss: 2.9258, Val Loss: 3.0731
Train Acc: 0.3034, Val Acc: 0.2802
Epoch 20/50, Train Loss: 2.0930, Val Loss: 2.6243
Train Acc: 0.4913, Val Acc: 0.3571
Epoch 30/50, Train Loss: 1.6212, Val Loss: 2.4390
Train Acc: 0.6095, Val Acc: 0.3919
Epoch 40/50, Train Loss: 1.2941, Val Loss: 2.3766
Train Acc: 0.6792, Val Acc: 0.4212
Epoch 50/50, Train Loss: 1.0533, Val Loss: 2.3590
Train Acc: 0.7278, Val Acc: 0.4377
Validation Accuracy: 0.4377
Testing: Layers=[256, 128], Activation=sigmoid, LR=0.001, Optimizer=sgd
Epoch 10/50, Train Loss: 4.1372, Val Loss: 4.1279
Train Acc: 0.0165, Val Acc: 0.0147
Early stopping at epoch 11
Validation Accuracy: 0.0147
Testing: Layers=[256, 128], Activation=sigmoid, LR=0.01, Optimizer=adam
Epoch 10/50, Train Loss: 1.5976, Val Loss: 2.4343
Train Acc: 0.5431, Val Acc: 0.4066
Early stopping at epoch 17
Validation Accuracy: 0.3956

Testing: Layers=[256, 128], Activation=sigmoid, LR=0.01, Optimizer=sgd
Epoch 10/50, Train Loss: 4.1363, Val Loss: 4.1200
Train Acc: 0.0170, Val Acc: 0.0220
Epoch 20/50, Train Loss: 4.1017, Val Loss: 4.0950
Train Acc: 0.0252, Val Acc: 0.0513
Epoch 30/50, Train Loss: 3.9461, Val Loss: 3.9469
Train Acc: 0.0619, Val Acc: 0.0806
Epoch 40/50, Train Loss: 3.5586, Val Loss: 3.5856
Train Acc: 0.0949, Val Acc: 0.1282
Epoch 50/50, Train Loss: 3.2779, Val Loss: 3.3569
Train Acc: 0.1485, Val Acc: 0.1538
Validation Accuracy: 0.1538
Testing: Layers=[256, 128], Activation=sigmoid, LR=0.1, Optimizer=adam
Early stopping at epoch 8
Validation Accuracy: 0.0165
Testing: Layers=[256, 128], Activation=sigmoid, LR=0.1, Optimizer=sgd
Epoch 10/50, Train Loss: 3.2143, Val Loss: 3.2338
Train Acc: 0.1462, Val Acc: 0.1722
Epoch 20/50, Train Loss: 2.0835, Val Loss: 2.5829
Train Acc: 0.4280, Val Acc: 0.3480
Epoch 30/50, Train Loss: 1.3714, Val Loss: 2.3457
Train Acc: 0.6214, Val Acc: 0.4048
Epoch 40/50, Train Loss: 0.9552, Val Loss: 2.3338
Train Acc: 0.7172, Val Acc: 0.4560
Early stopping at epoch 47
Validation Accuracy: 0.4487
Testing: Layers=[256, 128], Activation=tanh, LR=0.001, Optimizer=adam
Epoch 10/50, Train Loss: 1.4579, Val Loss: 2.4338
Train Acc: 0.6645, Val Acc: 0.4048
Epoch 20/50, Train Loss: 0.7675, Val Loss: 2.3468
Train Acc: 0.8286, Val Acc: 0.4359
Early stopping at epoch 24
Validation Accuracy: 0.4414
Testing: Layers=[256, 128], Activation=tanh, LR=0.001, Optimizer=sgd
Epoch 10/50, Train Loss: 3.8824, Val Loss: 3.9488
Train Acc: 0.1677, Val Acc: 0.1062
Epoch 20/50, Train Loss: 3.5127, Val Loss: 3.6579
Train Acc: 0.2406, Val Acc: 0.1538
Epoch 30/50, Train Loss: 3.1481, Val Loss: 3.3616
Train Acc: 0.2993, Val Acc: 0.2051
Epoch 40/50, Train Loss: 2.8537, Val Loss: 3.1397
Train Acc: 0.3662, Val Acc: 0.2857
Epoch 50/50, Train Loss: 2.6138, Val Loss: 2.9762
Train Acc: 0.4152, Val Acc: 0.3187
Validation Accuracy: 0.3187
Testing: Layers=[256, 128], Activation=tanh, LR=0.01, Optimizer=adam
Epoch 10/50, Train Loss: 1.2750, Val Loss: 2.6627
Train Acc: 0.6288, Val Acc: 0.3974
Early stopping at epoch 18
Validation Accuracy: 0.4029
Testing: Layers=[256, 128], Activation=tanh, LR=0.01, Optimizer=sgd
Epoch 10/50, Train Loss: 2.0823, Val Loss: 2.6123
Train Acc: 0.5055, Val Acc: 0.3810
Epoch 20/50, Train Loss: 1.3889, Val Loss: 2.4783

Train Acc: 0.6590, Val Acc: 0.4048
Epoch 30/50, Train Loss: 0.9869, Val Loss: 2.4654
Train Acc: 0.7631, Val Acc: 0.3993
Early stopping at epoch 31
Validation Accuracy: 0.3993
Testing: Layers=[256, 128], Activation=tanh, LR=0.1, Optimizer=adam
Epoch 10/50, Train Loss: 8.6506, Val Loss: 7.6394
Train Acc: 0.1196, Val Acc: 0.1538
Early stopping at epoch 17
Validation Accuracy: 0.1410
Testing: Layers=[256, 128], Activation=tanh, LR=0.1, Optimizer=sgd
Epoch 10/50, Train Loss: 0.9904, Val Loss: 2.9271
Train Acc: 0.7053, Val Acc: 0.3974
Early stopping at epoch 16
Validation Accuracy: 0.3993
Tuning completed!
Best validation accuracy: 0.4835
Best parameters: {'hidden_layers': [256, 128], 'activation': 'relu', 'learning_rate': 0.001, 'optimizer': 'adam'}



Training best model with optimal parameters...

Epoch 10/100, Train Loss: 1.2180, Val Loss: 2.4536

Train Acc: 0.6563, Val Acc: 0.4304

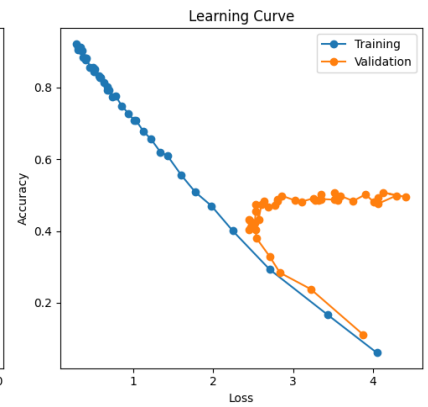
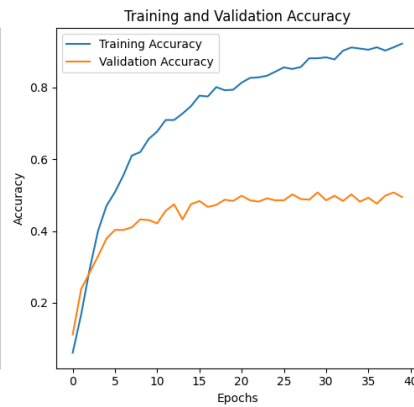
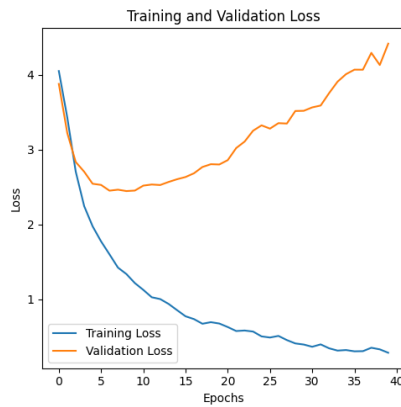
Epoch 20/100, Train Loss: 0.6776, Val Loss: 2.8020

Train Acc: 0.7933, Val Acc: 0.4835

Epoch 30/100, Train Loss: 0.3969, Val Loss: 3.5192

Train Acc: 0.8808, Val Acc: 0.5073

Early stopping at epoch 40

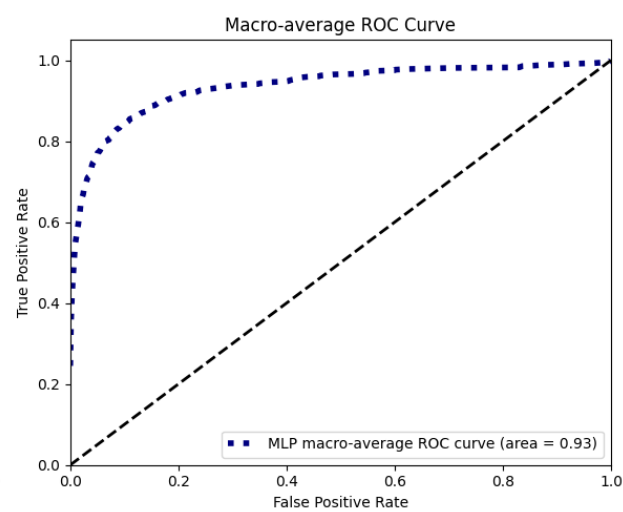
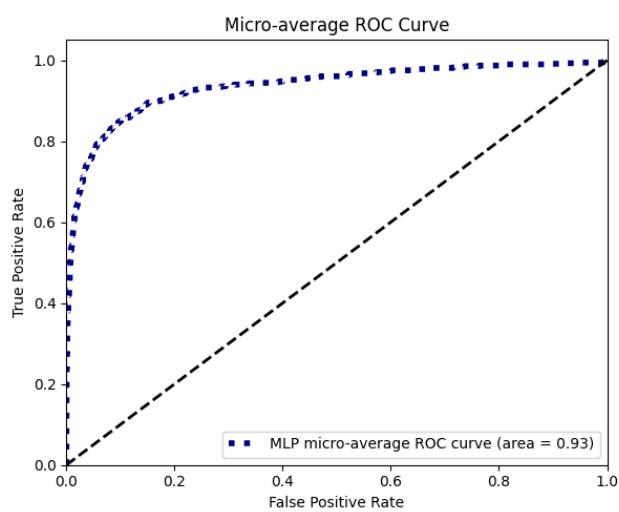
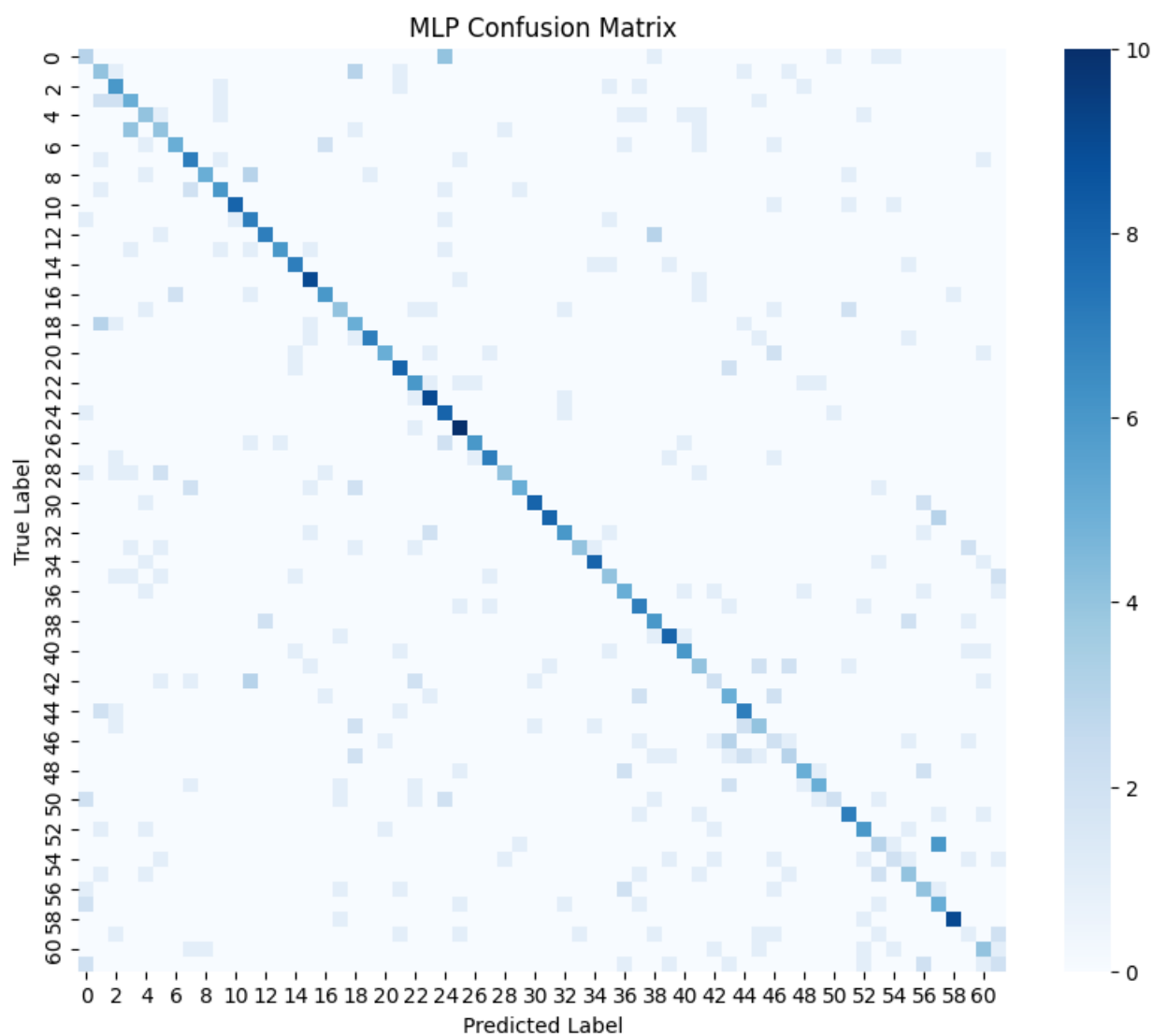


MLP Test Accuracy: 0.4971

MLP Test Precision: 0.5204

MLP Test Recall: 0.4971

MLP Test F1-Score: 0.4962



Classification Report for MLP:

precision recall f1-score support

0	0.23	0.27	0.25	11
1	0.27	0.36	0.31	11
2	0.38	0.55	0.44	11
3	0.38	0.45	0.42	11
4	0.33	0.36	0.35	11
5	0.33	0.36	0.35	11
6	0.71	0.45	0.56	11
7	0.50	0.64	0.56	11
8	0.83	0.45	0.59	11
9	0.55	0.55	0.55	11
A	0.89	0.73	0.80	11
B	0.44	0.64	0.52	11
C	0.78	0.64	0.70	11
D	0.86	0.55	0.67	11
E	0.64	0.64	0.64	11
F	0.60	0.82	0.69	11
G	0.60	0.55	0.57	11
H	0.44	0.36	0.40	11
I	0.29	0.45	0.36	11
J	0.88	0.64	0.74	11
K	0.71	0.45	0.56	11
L	0.57	0.73	0.64	11
M	0.43	0.55	0.48	11
N	0.60	0.82	0.69	11
O	0.42	0.73	0.53	11
P	0.59	0.91	0.71	11
Q	0.75	0.55	0.63	11
R	0.70	0.64	0.67	11
S	0.67	0.36	0.47	11
T	0.71	0.45	0.56	11
U	0.80	0.73	0.76	11
V	0.89	0.73	0.80	11
W	0.60	0.55	0.57	11
X	0.80	0.36	0.50	11
Y	0.73	0.73	0.73	11
Z	0.44	0.36	0.40	11
a	0.42	0.45	0.43	11
b	0.47	0.64	0.54	11
c	0.43	0.55	0.48	11
d	0.62	0.73	0.67	11
e	0.60	0.55	0.57	11
f	0.33	0.36	0.35	11
g	0.29	0.18	0.22	11
h	0.33	0.45	0.38	11
i	0.54	0.64	0.58	11
j	0.36	0.36	0.36	11
k	0.15	0.18	0.17	11
l	0.38	0.27	0.32	11
m	0.56	0.45	0.50	11
n	0.62	0.45	0.53	11
o	0.50	0.18	0.27	11

```

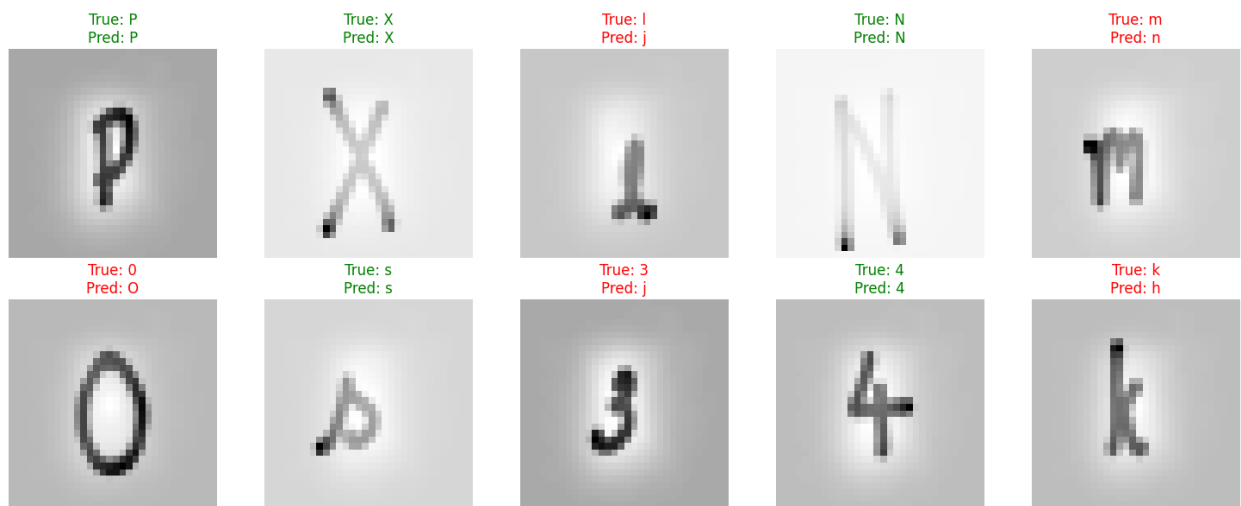
p 0.58 0.64 0.61 11
q 0.50 0.55 0.52 11
r 0.27 0.27 0.27 11
s 0.33 0.18 0.24 11
t 0.36 0.36 0.36 11
u 0.33 0.36 0.35 11
v 0.31 0.45 0.37 11
w 0.90 0.82 0.86 11
x 0.14 0.09 0.11 11
y 0.36 0.36 0.36 11
z 0.22 0.18 0.20 11

```

```

accuracy 0.50 682
macro avg 0.52 0.50 0.50 682
weighted avg 0.52 0.50 0.50 682

```



4. Hyperparameter Tuning

In[8]:

```

# Define hyperparameters to tune
hidden_layers_options=[[128],[256],[128,64],[256,128]]
activation_options=['relu','sigmoid','tanh']
learning_rates=[0.001,0.01,0.1]
optimizers_list=['adam','sgd']

# Tuning loop
best_val_acc=0
best_params={}
best_model=None
all_results=[]

print("Starting hyperparameter tuning...")

for hidden_layers in hidden_layers_options:
    for activation in activation_options:
        for learning_rate in learning_rates:

```

```

for opt_name in optimizers_list:
    print(f"Testing: Layers={hidden_layers}, Activation={activation}

    # Initialize model
    model=MLP(input_size,hidden_layers,num_classes,activation=

    # Define loss and optimizer
    criterion=nn.CrossEntropyLoss()

    if opt_name=='adam':
        optimizer=optim.Adam(model.parameters(),lr=lr)
    else:
        optimizer=optim.SGD(model.parameters(),lr=lr,momentum=0.9

    # Train model
    model,train_losses,val_losses,train_accs,val_accs=train_model
        model,train_loader,val_loader,criterion,optimizer,
        num_epochs=50,patience=7
    )

    # Evaluate on validation set
    val_acc,_,_,_=evaluate_model(model,val_loader)

    print(f"Validation Accuracy:{val_acc:.4f}")

    # Store results
    result={
        'hidden_layers':hidden_layers,
        'activation':activation,
        'learning_rate':lr,
        'optimizer':opt_name,
        'val_accuracy':val_acc,
        'train_losses':train_losses,
        'val_losses':val_losses,
        'train_accs':train_accs,
        'val_accs':val_accs
    }
    all_results.append(result)

    # Update best model
    if val_acc>best_val_acc:
        best_val_acc=val_acc
        best_params={
            'hidden_layers':hidden_layers,
            'activation':activation,
            'learning_rate':lr,
            'optimizer':opt_name
        }
        best_model=model
        print("New best model found!")

print("Tuning completed!")
print(f"Best validation accuracy:{best_val_acc:.4f}")

```

```
print(f"Best parameters:{best_params}")

# Plot training curves for all configurations
fig,axes=plt.subplots(2,2,figsize=(15,12))
axes=axes.flatten()

for i,result in enumerate(all_results):
    axes[0].plot(result['train_losses'],label=f"Config{i+1}")
    axes[1].plot(result['val_losses'],label=f"Config{i+1}")
    axes[2].plot(result['train_accs'],label=f"Config{i+1}")
    axes[3].plot(result['val_accs'],label=f"Config{i+1}")

axes[0].set_title('Training Loss')
axes[0].set_xlabel('Epoch')
axes[0].set_ylabel('Loss')
axes[0].legend()

axes[1].set_title('Validation Loss')
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Loss')
axes[1].legend()

axes[2].set_title('Training Accuracy')
axes[2].set_xlabel('Epoch')
axes[2].set_ylabel('Accuracy')
axes[2].legend()

axes[3].set_title('Validation Accuracy')
axes[3].set_xlabel('Epoch')
axes[3].set_ylabel('Accuracy')
axes[3].legend()

plt.tight_layout()
plt.show()
```

Starting hyperparameter tuning...

Testing: Layers=[128], Activation=relu, LR=0.001, Optimizer=adam
Epoch 10/50, Train Loss: 1.3568, Val Loss: 2.5266
Train Acc: 0.6554, Val Acc: 0.4066
Epoch 20/50, Train Loss: 0.8028, Val Loss: 2.6862
Train Acc: 0.7933, Val Acc: 0.4322
Epoch 30/50, Train Loss: 0.5172, Val Loss: 2.9831
Train Acc: 0.8731, Val Acc: 0.4304
Early stopping at epoch 31
Validation Accuracy: 0.4341
New best model found!

Testing: Layers=[128], Activation=relu, LR=0.001, Optimizer=sgd
Epoch 10/50, Train Loss: 3.7996, Val Loss: 3.9883
Train Acc: 0.2021, Val Acc: 0.1300
Epoch 20/50, Train Loss: 3.3882, Val Loss: 3.7126
Train Acc: 0.2896, Val Acc: 0.2088
Epoch 30/50, Train Loss: 2.9282, Val Loss: 3.4051
Train Acc: 0.3804, Val Acc: 0.2674
Epoch 40/50, Train Loss: 2.5434, Val Loss: 3.1461
Train Acc: 0.4363, Val Acc: 0.2894
Epoch 50/50, Train Loss: 2.2474, Val Loss: 2.9547
Train Acc: 0.4973, Val Acc: 0.3278
Validation Accuracy: 0.3278

Testing: Layers=[128], Activation=relu, LR=0.01, Optimizer=adam
Epoch 10/50, Train Loss: 1.3863, Val Loss: 13.8891
Train Acc: 0.6581, Val Acc: 0.3718
Early stopping at epoch 18
Validation Accuracy: 0.3681

Testing: Layers=[128], Activation=relu, LR=0.01, Optimizer=sgd
Epoch 10/50, Train Loss: 1.7237, Val Loss: 2.6127
Train Acc: 0.5623, Val Acc: 0.3919
Epoch 20/50, Train Loss: 1.0278, Val Loss: 2.5921
Train Acc: 0.7351, Val Acc: 0.4432
Epoch 30/50, Train Loss: 0.7444, Val Loss: 2.7064
Train Acc: 0.7988, Val Acc: 0.4560
Early stopping at epoch 33
Validation Accuracy: 0.4359
New best model found!

Testing: Layers=[128], Activation=relu, LR=0.1, Optimizer=adam
Epoch 10/50, Train Loss: 110.4365, Val Loss: 1379.4136
Train Acc: 0.3616, Val Acc: 0.1960
Epoch 20/50, Train Loss: 82.3961, Val Loss: 2214.3931
Train Acc: 0.5142, Val Acc: 0.2582
Early stopping at epoch 30
Validation Accuracy: 0.2711

Testing: Layers=[128], Activation=relu, LR=0.1, Optimizer=sgd
Early stopping at epoch 8
Validation Accuracy: 0.1154

Testing: Layers=[128], Activation=sigmoid, LR=0.001, Optimizer=adam
Epoch 10/50, Train Loss: 2.5772, Val Loss: 2.9294
Train Acc: 0.4519, Val Acc: 0.3333
Epoch 20/50, Train Loss: 1.8808, Val Loss: 2.5920
Train Acc: 0.5742, Val Acc: 0.3901
Epoch 30/50, Train Loss: 1.4801, Val Loss: 2.4949

Train Acc: 0.6622, Val Acc: 0.3974
Early stopping at epoch 31
Validation Accuracy: 0.3993
Testing: Layers=[128], Activation=sigmoid, LR=0.001, Optimizer=sgd
Epoch 10/50, Train Loss: 4.0848, Val Loss: 4.0989
Train Acc: 0.0385, Val Acc: 0.0714
Epoch 20/50, Train Loss: 4.0318, Val Loss: 4.0576
Train Acc: 0.0710, Val Acc: 0.1136
Epoch 30/50, Train Loss: 3.9628, Val Loss: 4.0096
Train Acc: 0.1040, Val Acc: 0.1410
Epoch 40/50, Train Loss: 3.8854, Val Loss: 3.9498
Train Acc: 0.1512, Val Acc: 0.1703
Early stopping at epoch 44
Validation Accuracy: 0.1667
Testing: Layers=[128], Activation=sigmoid, LR=0.01, Optimizer=adam
Epoch 10/50, Train Loss: 0.9554, Val Loss: 2.5994
Train Acc: 0.7566, Val Acc: 0.4011
Early stopping at epoch 20
Validation Accuracy: 0.3828
Testing: Layers=[128], Activation=sigmoid, LR=0.01, Optimizer=sgd
Epoch 10/50, Train Loss: 3.4110, Val Loss: 3.4990
Train Acc: 0.2392, Val Acc: 0.2454
Epoch 20/50, Train Loss: 2.6810, Val Loss: 2.9560
Train Acc: 0.3960, Val Acc: 0.3407
Epoch 30/50, Train Loss: 2.2578, Val Loss: 2.6843
Train Acc: 0.4730, Val Acc: 0.3718
Epoch 40/50, Train Loss: 1.9951, Val Loss: 2.5547
Train Acc: 0.5284, Val Acc: 0.3956
Epoch 50/50, Train Loss: 1.8105, Val Loss: 2.4875
Train Acc: 0.5646, Val Acc: 0.4121
Validation Accuracy: 0.4121
Testing: Layers=[128], Activation=sigmoid, LR=0.1, Optimizer=adam
Epoch 10/50, Train Loss: 3.7445, Val Loss: 4.9800
Train Acc: 0.3236, Val Acc: 0.2601
Early stopping at epoch 18
Validation Accuracy: 0.2308
Testing: Layers=[128], Activation=sigmoid, LR=0.1, Optimizer=sgd
Epoch 10/50, Train Loss: 1.5295, Val Loss: 2.5475
Train Acc: 0.5976, Val Acc: 0.3773
Early stopping at epoch 15
Validation Accuracy: 0.3974
Testing: Layers=[128], Activation=tanh, LR=0.001, Optimizer=adam
Epoch 10/50, Train Loss: 1.5947, Val Loss: 2.5620
Train Acc: 0.6357, Val Acc: 0.3755
Epoch 20/50, Train Loss: 1.0242, Val Loss: 2.5752
Train Acc: 0.7704, Val Acc: 0.3956
Early stopping at epoch 28
Validation Accuracy: 0.3919
Testing: Layers=[128], Activation=tanh, LR=0.001, Optimizer=sgd
Epoch 10/50, Train Loss: 3.5955, Val Loss: 3.7658
Train Acc: 0.2626, Val Acc: 0.1722
Epoch 20/50, Train Loss: 3.1099, Val Loss: 3.4106
Train Acc: 0.3666, Val Acc: 0.2454
Epoch 30/50, Train Loss: 2.7511, Val Loss: 3.1516

Train Acc: 0.4349, Val Acc: 0.3004
Epoch 40/50, Train Loss: 2.4718, Val Loss: 2.9770
Train Acc: 0.4817, Val Acc: 0.3315
Epoch 50/50, Train Loss: 2.2646, Val Loss: 2.8547
Train Acc: 0.5261, Val Acc: 0.3443
Validation Accuracy: 0.3443
Testing: Layers=[128], Activation=tanh, LR=0.01, Optimizer=adam
Epoch 10/50, Train Loss: 0.6697, Val Loss: 2.9709
Train Acc: 0.8112, Val Acc: 0.3810
Early stopping at epoch 18
Validation Accuracy: 0.3846
Testing: Layers=[128], Activation=tanh, LR=0.01, Optimizer=sgd
Epoch 10/50, Train Loss: 1.8341, Val Loss: 2.5868
Train Acc: 0.5871, Val Acc: 0.3755
Epoch 20/50, Train Loss: 1.2592, Val Loss: 2.4861
Train Acc: 0.7076, Val Acc: 0.4103
Epoch 30/50, Train Loss: 0.9220, Val Loss: 2.5402
Train Acc: 0.7956, Val Acc: 0.4194
Early stopping at epoch 37
Validation Accuracy: 0.4048
Testing: Layers=[128], Activation=tanh, LR=0.1, Optimizer=adam
Epoch 10/50, Train Loss: 7.5471, Val Loss: 8.3817
Train Acc: 0.2819, Val Acc: 0.2473
Epoch 20/50, Train Loss: 7.2077, Val Loss: 9.0838
Train Acc: 0.3396, Val Acc: 0.2546
Early stopping at epoch 29
Validation Accuracy: 0.2711
Testing: Layers=[128], Activation=tanh, LR=0.1, Optimizer=sgd
Epoch 10/50, Train Loss: 0.7293, Val Loss: 3.0935
Train Acc: 0.7910, Val Acc: 0.3773
Epoch 20/50, Train Loss: 0.2653, Val Loss: 3.5580
Train Acc: 0.9203, Val Acc: 0.3700
Early stopping at epoch 26
Validation Accuracy: 0.3718
Testing: Layers=[256], Activation=relu, LR=0.001, Optimizer=adam
Epoch 10/50, Train Loss: 0.9426, Val Loss: 2.5837
Train Acc: 0.7599, Val Acc: 0.4341
Epoch 20/50, Train Loss: 0.4210, Val Loss: 2.8679
Train Acc: 0.8964, Val Acc: 0.4560
Early stopping at epoch 25
Validation Accuracy: 0.4542
New best model found!
Testing: Layers=[256], Activation=relu, LR=0.001, Optimizer=sgd
Epoch 10/50, Train Loss: 3.7654, Val Loss: 4.0071
Train Acc: 0.2305, Val Acc: 0.1484
Epoch 20/50, Train Loss: 3.3221, Val Loss: 3.6961
Train Acc: 0.3336, Val Acc: 0.2234
Epoch 30/50, Train Loss: 2.8410, Val Loss: 3.3631
Train Acc: 0.4258, Val Acc: 0.2729
Epoch 40/50, Train Loss: 2.4327, Val Loss: 3.0790
Train Acc: 0.4808, Val Acc: 0.3297
Epoch 50/50, Train Loss: 2.1240, Val Loss: 2.8828
Train Acc: 0.5302, Val Acc: 0.3480
Validation Accuracy: 0.3480

Testing: Layers=[256], Activation=relu, LR=0.01, Optimizer=adam
Epoch 10/50, Train Loss: 2.1614, Val Loss: 27.6184
Train Acc: 0.6682, Val Acc: 0.3663
Epoch 20/50, Train Loss: 1.3476, Val Loss: 48.1619
Train Acc: 0.8245, Val Acc: 0.3718
Early stopping at epoch 21
Validation Accuracy: 0.3791
Testing: Layers=[256], Activation=relu, LR=0.01, Optimizer=sgd
Epoch 10/50, Train Loss: 1.5356, Val Loss: 2.6083
Train Acc: 0.6059, Val Acc: 0.3938
Epoch 20/50, Train Loss: 0.8388, Val Loss: 2.6015
Train Acc: 0.7878, Val Acc: 0.4432
Epoch 30/50, Train Loss: 0.5046, Val Loss: 2.7137
Train Acc: 0.8827, Val Acc: 0.4652
Epoch 40/50, Train Loss: 0.3425, Val Loss: 2.8356
Train Acc: 0.9340, Val Acc: 0.4597
Early stopping at epoch 41
Validation Accuracy: 0.4597
New best model found!
Testing: Layers=[256], Activation=relu, LR=0.1, Optimizer=adam
Epoch 10/50, Train Loss: 232.3328, Val Loss: 2447.6903
Train Acc: 0.5119, Val Acc: 0.3040
Epoch 20/50, Train Loss: 198.3365, Val Loss: 4721.0708
Train Acc: 0.6755, Val Acc: 0.3095
Early stopping at epoch 25
Validation Accuracy: 0.3297
Testing: Layers=[256], Activation=relu, LR=0.1, Optimizer=sgd
Early stopping at epoch 9
Validation Accuracy: 0.1190
Testing: Layers=[256], Activation=sigmoid, LR=0.001, Optimizer=adam
Epoch 10/50, Train Loss: 2.1515, Val Loss: 2.6720
Train Acc: 0.5270, Val Acc: 0.3810
Epoch 20/50, Train Loss: 1.4785, Val Loss: 2.4479
Train Acc: 0.6599, Val Acc: 0.4231
Early stopping at epoch 27
Validation Accuracy: 0.4121
Testing: Layers=[256], Activation=sigmoid, LR=0.001, Optimizer=sgd
Epoch 10/50, Train Loss: 4.0844, Val Loss: 4.0850
Train Acc: 0.0339, Val Acc: 0.0385
Epoch 20/50, Train Loss: 4.0160, Val Loss: 4.0356
Train Acc: 0.0765, Val Acc: 0.1245
Epoch 30/50, Train Loss: 3.9407, Val Loss: 3.9762
Train Acc: 0.1169, Val Acc: 0.1538
Early stopping at epoch 35
Validation Accuracy: 0.1667
Testing: Layers=[256], Activation=sigmoid, LR=0.01, Optimizer=adam
Epoch 10/50, Train Loss: 0.5541, Val Loss: 2.7599
Train Acc: 0.8607, Val Acc: 0.4048
Epoch 20/50, Train Loss: 0.1656, Val Loss: 3.1011
Train Acc: 0.9688, Val Acc: 0.4121
Epoch 30/50, Train Loss: 0.0989, Val Loss: 3.3861
Train Acc: 0.9803, Val Acc: 0.4121
Early stopping at epoch 31
Validation Accuracy: 0.4249

Testing: Layers=[256], Activation=sigmoid, LR=0.01, Optimizer=sgd
Epoch 10/50, Train Loss: 3.2937, Val Loss: 3.3894
Train Acc: 0.2273, Val Acc: 0.2674
Epoch 20/50, Train Loss: 2.4960, Val Loss: 2.8152
Train Acc: 0.4125, Val Acc: 0.3498
Epoch 30/50, Train Loss: 2.0595, Val Loss: 2.5838
Train Acc: 0.4995, Val Acc: 0.3828
Epoch 40/50, Train Loss: 1.8383, Val Loss: 2.5034
Train Acc: 0.5582, Val Acc: 0.4029
Epoch 50/50, Train Loss: 1.6541, Val Loss: 2.4778
Train Acc: 0.5898, Val Acc: 0.4066
Validation Accuracy: 0.4066
Testing: Layers=[256], Activation=sigmoid, LR=0.1, Optimizer=adam
Epoch 10/50, Train Loss: 5.6924, Val Loss: 8.0678
Train Acc: 0.3391, Val Acc: 0.2015
Epoch 20/50, Train Loss: 5.1853, Val Loss: 9.0328
Train Acc: 0.4102, Val Acc: 0.2344
Early stopping at epoch 23
Validation Accuracy: 0.2454
Testing: Layers=[256], Activation=sigmoid, LR=0.1, Optimizer=sgd
Epoch 10/50, Train Loss: 1.4229, Val Loss: 2.7302
Train Acc: 0.6031, Val Acc: 0.3810
Epoch 20/50, Train Loss: 0.7174, Val Loss: 2.8332
Train Acc: 0.7988, Val Acc: 0.3846
Early stopping at epoch 30
Validation Accuracy: 0.4249
Testing: Layers=[256], Activation=tanh, LR=0.001, Optimizer=adam
Epoch 10/50, Train Loss: 1.2822, Val Loss: 2.5690
Train Acc: 0.6994, Val Acc: 0.3993
Epoch 20/50, Train Loss: 0.6970, Val Loss: 2.6785
Train Acc: 0.8607, Val Acc: 0.4048
Early stopping at epoch 24
Validation Accuracy: 0.3938
Testing: Layers=[256], Activation=tanh, LR=0.001, Optimizer=sgd
Epoch 10/50, Train Loss: 3.4791, Val Loss: 3.6465
Train Acc: 0.2956, Val Acc: 0.2106
Epoch 20/50, Train Loss: 2.9029, Val Loss: 3.2564
Train Acc: 0.4056, Val Acc: 0.2967
Epoch 30/50, Train Loss: 2.5167, Val Loss: 3.0072
Train Acc: 0.4720, Val Acc: 0.3132
Epoch 40/50, Train Loss: 2.2482, Val Loss: 2.8420
Train Acc: 0.5202, Val Acc: 0.3425
Epoch 50/50, Train Loss: 2.0434, Val Loss: 2.7388
Train Acc: 0.5610, Val Acc: 0.3626
Validation Accuracy: 0.3626
Testing: Layers=[256], Activation=tanh, LR=0.01, Optimizer=adam
Epoch 10/50, Train Loss: 0.5025, Val Loss: 3.2236
Train Acc: 0.8451, Val Acc: 0.4011
Early stopping at epoch 17
Validation Accuracy: 0.3626
Testing: Layers=[256], Activation=tanh, LR=0.01, Optimizer=sgd
Epoch 10/50, Train Loss: 1.6550, Val Loss: 2.6190
Train Acc: 0.6205, Val Acc: 0.3828
Epoch 20/50, Train Loss: 1.1070, Val Loss: 2.6584

Train Acc: 0.7539, Val Acc: 0.4029
Epoch 30/50, Train Loss: 0.7751, Val Loss: 2.7618
Train Acc: 0.8332, Val Acc: 0.4066
Early stopping at epoch 31
Validation Accuracy: 0.3938
Testing: Layers=[256], Activation=tanh, LR=0.1, Optimizer=adam
Epoch 10/50, Train Loss: 11.8644, Val Loss: 16.0947
Train Acc: 0.3194, Val Acc: 0.2253
Early stopping at epoch 18
Validation Accuracy: 0.2473
Testing: Layers=[256], Activation=tanh, LR=0.1, Optimizer=sgd
Epoch 10/50, Train Loss: 0.5615, Val Loss: 3.6191
Train Acc: 0.8323, Val Acc: 0.3919
Epoch 20/50, Train Loss: 0.1642, Val Loss: 4.0314
Train Acc: 0.9542, Val Acc: 0.3956
Early stopping at epoch 24
Validation Accuracy: 0.4029
Testing: Layers=[128, 64], Activation=relu, LR=0.001, Optimizer=adam
Epoch 10/50, Train Loss: 1.8313, Val Loss: 2.4735
Train Acc: 0.4991, Val Acc: 0.3993
Epoch 20/50, Train Loss: 1.1932, Val Loss: 2.4374
Train Acc: 0.6572, Val Acc: 0.4762
Epoch 30/50, Train Loss: 0.9590, Val Loss: 2.8934
Train Acc: 0.7177, Val Acc: 0.4890
Epoch 40/50, Train Loss: 0.7619, Val Loss: 3.2747
Train Acc: 0.7718, Val Acc: 0.5147
Early stopping at epoch 47
Validation Accuracy: 0.4890
New best model found!
Testing: Layers=[128, 64], Activation=relu, LR=0.001, Optimizer=sgd
Epoch 10/50, Train Loss: 4.0880, Val Loss: 4.1031
Train Acc: 0.0463, Val Acc: 0.0330
Epoch 20/50, Train Loss: 4.0126, Val Loss: 4.0505
Train Acc: 0.0852, Val Acc: 0.0678
Epoch 30/50, Train Loss: 3.8790, Val Loss: 3.9448
Train Acc: 0.1095, Val Acc: 0.0916
Epoch 40/50, Train Loss: 3.6548, Val Loss: 3.7913
Train Acc: 0.1434, Val Acc: 0.1081
Epoch 50/50, Train Loss: 3.3963, Val Loss: 3.5859
Train Acc: 0.1760, Val Acc: 0.1538
Validation Accuracy: 0.1538
Testing: Layers=[128, 64], Activation=relu, LR=0.01, Optimizer=adam
Epoch 10/50, Train Loss: 2.2457, Val Loss: 2.8759
Train Acc: 0.3717, Val Acc: 0.3407
Early stopping at epoch 14
Validation Accuracy: 0.3370
Testing: Layers=[128, 64], Activation=relu, LR=0.01, Optimizer=sgd
Epoch 10/50, Train Loss: 2.5199, Val Loss: 2.8462
Train Acc: 0.3300, Val Acc: 0.3223
Epoch 20/50, Train Loss: 1.5281, Val Loss: 2.5264
Train Acc: 0.5550, Val Acc: 0.4304
Epoch 30/50, Train Loss: 1.1612, Val Loss: 2.7480
Train Acc: 0.6544, Val Acc: 0.4432
Epoch 40/50, Train Loss: 0.8643, Val Loss: 2.8208

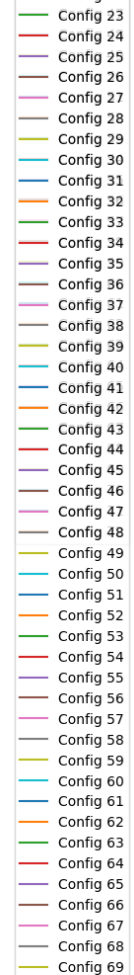
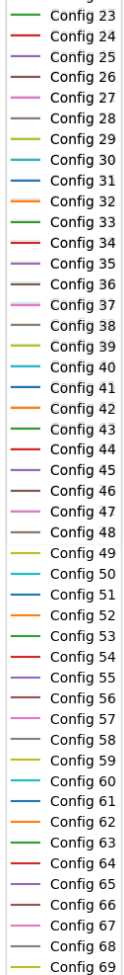
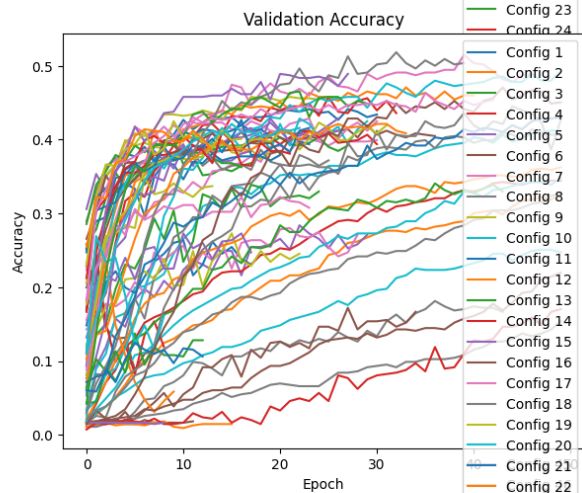
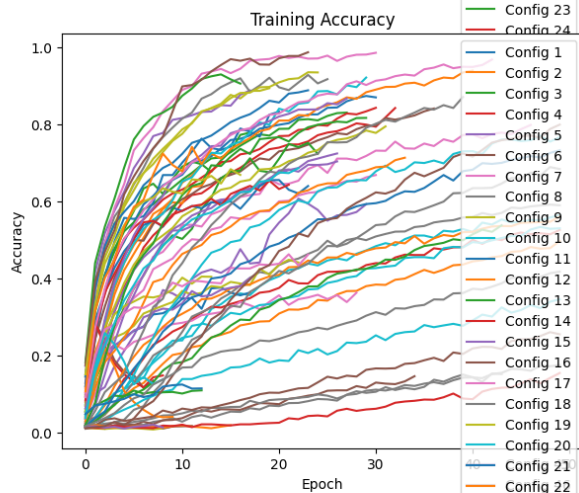
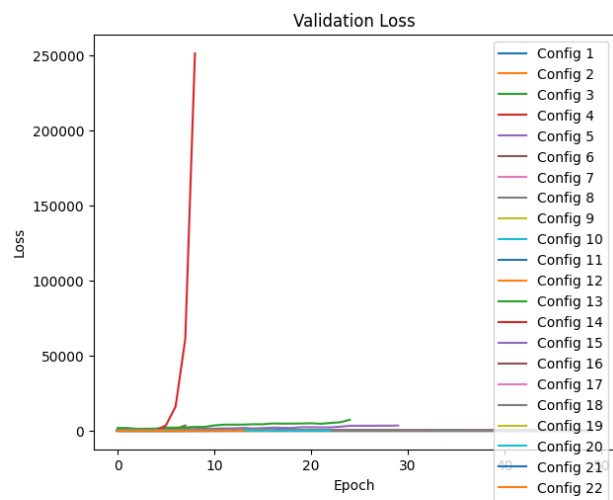
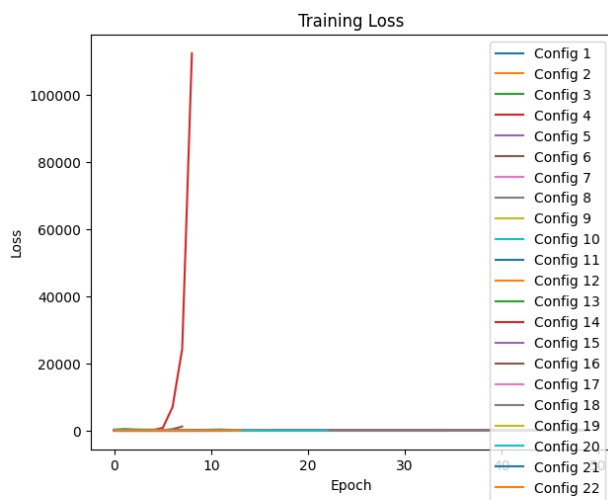
Train Acc: 0.7296, Val Acc: 0.4744
Epoch 50/50, Train Loss: 0.7023, Val Loss: 3.0985
Train Acc: 0.7663, Val Acc: 0.4927
Validation Accuracy: 0.4927
New best model found!
Testing: Layers=[128, 64], Activation=relu, LR=0.1, Optimizer=adam
Early stopping at epoch 8
Validation Accuracy: 0.0165
Testing: Layers=[128, 64], Activation=relu, LR=0.1, Optimizer=sgd
Early stopping at epoch 10
Validation Accuracy: 0.0586
Testing: Layers=[128, 64], Activation=sigmoid, LR=0.001, Optimizer=adam
Epoch 10/50, Train Loss: 3.4127, Val Loss: 3.4613
Train Acc: 0.1797, Val Acc: 0.1685
Epoch 20/50, Train Loss: 2.7337, Val Loss: 3.0115
Train Acc: 0.3272, Val Acc: 0.2564
Epoch 30/50, Train Loss: 2.2868, Val Loss: 2.7706
Train Acc: 0.4253, Val Acc: 0.3150
Epoch 40/50, Train Loss: 1.9773, Val Loss: 2.6395
Train Acc: 0.4991, Val Acc: 0.3315
Early stopping at epoch 44
Validation Accuracy: 0.3297
Testing: Layers=[128, 64], Activation=sigmoid, LR=0.001, Optimizer=sgd
Early stopping at epoch 10
Validation Accuracy: 0.0165
Testing: Layers=[128, 64], Activation=sigmoid, LR=0.01, Optimizer=adam
Epoch 10/50, Train Loss: 1.7233, Val Loss: 2.4702
Train Acc: 0.5243, Val Acc: 0.3608
Epoch 20/50, Train Loss: 1.1756, Val Loss: 2.5247
Train Acc: 0.6563, Val Acc: 0.4286
Early stopping at epoch 27
Validation Accuracy: 0.4286
Testing: Layers=[128, 64], Activation=sigmoid, LR=0.01, Optimizer=sgd
Epoch 10/50, Train Loss: 4.1325, Val Loss: 4.1210
Train Acc: 0.0170, Val Acc: 0.0165
Early stopping at epoch 12
Validation Accuracy: 0.0183
Testing: Layers=[128, 64], Activation=sigmoid, LR=0.1, Optimizer=adam
Early stopping at epoch 8
Validation Accuracy: 0.0165
Testing: Layers=[128, 64], Activation=sigmoid, LR=0.1, Optimizer=sgd
Epoch 10/50, Train Loss: 3.1214, Val Loss: 3.1489
Train Acc: 0.1792, Val Acc: 0.1978
Epoch 20/50, Train Loss: 2.1932, Val Loss: 2.6465
Train Acc: 0.3983, Val Acc: 0.3168
Epoch 30/50, Train Loss: 1.7087, Val Loss: 2.4331
Train Acc: 0.5211, Val Acc: 0.3791
Epoch 40/50, Train Loss: 1.3121, Val Loss: 2.3683
Train Acc: 0.6219, Val Acc: 0.4359
Epoch 50/50, Train Loss: 1.0850, Val Loss: 2.3553
Train Acc: 0.6838, Val Acc: 0.4267
Validation Accuracy: 0.4267
Testing: Layers=[128, 64], Activation=tanh, LR=0.001, Optimizer=adam
Epoch 10/50, Train Loss: 2.0248, Val Loss: 2.5699

Train Acc: 0.5312, Val Acc: 0.3791
Epoch 20/50, Train Loss: 1.3370, Val Loss: 2.4108
Train Acc: 0.6842, Val Acc: 0.3974
Epoch 30/50, Train Loss: 0.9358, Val Loss: 2.4030
Train Acc: 0.7837, Val Acc: 0.4176
Early stopping at epoch 32
Validation Accuracy: 0.4194
Testing: Layers=[128, 64], Activation=tanh, LR=0.001, Optimizer=sgd
Epoch 10/50, Train Loss: 3.9263, Val Loss: 3.9673
Train Acc: 0.1123, Val Acc: 0.0897
Epoch 20/50, Train Loss: 3.6607, Val Loss: 3.7474
Train Acc: 0.1824, Val Acc: 0.1447
Epoch 30/50, Train Loss: 3.3745, Val Loss: 3.5128
Train Acc: 0.2365, Val Acc: 0.1905
Epoch 40/50, Train Loss: 3.1436, Val Loss: 3.3202
Train Acc: 0.2777, Val Acc: 0.2289
Epoch 50/50, Train Loss: 2.9320, Val Loss: 3.1680
Train Acc: 0.3465, Val Acc: 0.2473
Validation Accuracy: 0.2473
Testing: Layers=[128, 64], Activation=tanh, LR=0.01, Optimizer=adam
Epoch 10/50, Train Loss: 1.4762, Val Loss: 2.6209
Train Acc: 0.5839, Val Acc: 0.3718
Epoch 20/50, Train Loss: 1.1829, Val Loss: 2.6627
Train Acc: 0.6549, Val Acc: 0.3883
Early stopping at epoch 24
Validation Accuracy: 0.4158
Testing: Layers=[128, 64], Activation=tanh, LR=0.01, Optimizer=sgd
Epoch 10/50, Train Loss: 2.4586, Val Loss: 2.7904
Train Acc: 0.4349, Val Acc: 0.3535
Epoch 20/50, Train Loss: 1.7300, Val Loss: 2.4675
Train Acc: 0.5770, Val Acc: 0.4084
Epoch 30/50, Train Loss: 1.2895, Val Loss: 2.4173
Train Acc: 0.6797, Val Acc: 0.4158
Early stopping at epoch 34
Validation Accuracy: 0.4084
Testing: Layers=[128, 64], Activation=tanh, LR=0.1, Optimizer=adam
Epoch 10/50, Train Loss: 5.3720, Val Loss: 5.0990
Train Acc: 0.1063, Val Acc: 0.1081
Early stopping at epoch 13
Validation Accuracy: 0.1282
Testing: Layers=[128, 64], Activation=tanh, LR=0.1, Optimizer=sgd
Epoch 10/50, Train Loss: 1.5280, Val Loss: 2.6761
Train Acc: 0.5660, Val Acc: 0.3938
Epoch 20/50, Train Loss: 1.1550, Val Loss: 2.8822
Train Acc: 0.6535, Val Acc: 0.3846
Early stopping at epoch 22
Validation Accuracy: 0.3810
Testing: Layers=[256, 128], Activation=relu, LR=0.001, Optimizer=adam
Epoch 10/50, Train Loss: 1.2116, Val Loss: 2.5038
Train Acc: 0.6572, Val Acc: 0.4377
Epoch 20/50, Train Loss: 0.6280, Val Loss: 2.9819
Train Acc: 0.8080, Val Acc: 0.4542
Early stopping at epoch 28
Validation Accuracy: 0.4890

Testing: Layers=[256, 128], Activation=relu, LR=0.001, Optimizer=sgd
Epoch 10/50, Train Loss: 4.0703, Val Loss: 4.1363
Train Acc: 0.0522, Val Acc: 0.0421
Epoch 20/50, Train Loss: 3.9760, Val Loss: 4.0914
Train Acc: 0.1045, Val Acc: 0.0934
Epoch 30/50, Train Loss: 3.8153, Val Loss: 3.9758
Train Acc: 0.1636, Val Acc: 0.1374
Epoch 40/50, Train Loss: 3.5338, Val Loss: 3.7479
Train Acc: 0.2049, Val Acc: 0.1575
Epoch 50/50, Train Loss: 3.1413, Val Loss: 3.4110
Train Acc: 0.2557, Val Acc: 0.2198
Validation Accuracy: 0.2198
Testing: Layers=[256, 128], Activation=relu, LR=0.01, Optimizer=adam
Epoch 10/50, Train Loss: 2.2790, Val Loss: 3.5082
Train Acc: 0.3896, Val Acc: 0.3059
Epoch 20/50, Train Loss: 2.0625, Val Loss: 3.6913
Train Acc: 0.4349, Val Acc: 0.3132
Early stopping at epoch 24
Validation Accuracy: 0.3077
Testing: Layers=[256, 128], Activation=relu, LR=0.01, Optimizer=sgd
Epoch 10/50, Train Loss: 2.2349, Val Loss: 2.8016
Train Acc: 0.4083, Val Acc: 0.3260
Epoch 20/50, Train Loss: 1.1456, Val Loss: 2.4326
Train Acc: 0.6774, Val Acc: 0.4634
Epoch 30/50, Train Loss: 0.7138, Val Loss: 2.5865
Train Acc: 0.7906, Val Acc: 0.4908
Early stopping at epoch 40
Validation Accuracy: 0.5037
New best model found!
Testing: Layers=[256, 128], Activation=relu, LR=0.1, Optimizer=adam
Early stopping at epoch 9
Validation Accuracy: 0.0165
Testing: Layers=[256, 128], Activation=relu, LR=0.1, Optimizer=sgd
Early stopping at epoch 9
Validation Accuracy: 0.1612
Testing: Layers=[256, 128], Activation=sigmoid, LR=0.001, Optimizer=adam
Epoch 10/50, Train Loss: 2.9432, Val Loss: 3.0992
Train Acc: 0.2851, Val Acc: 0.2875
Epoch 20/50, Train Loss: 2.1276, Val Loss: 2.6668
Train Acc: 0.4826, Val Acc: 0.3498
Epoch 30/50, Train Loss: 1.6522, Val Loss: 2.5099
Train Acc: 0.5930, Val Acc: 0.3919
Epoch 40/50, Train Loss: 1.3091, Val Loss: 2.4234
Train Acc: 0.6783, Val Acc: 0.4139
Epoch 50/50, Train Loss: 1.0588, Val Loss: 2.4067
Train Acc: 0.7324, Val Acc: 0.4304
Validation Accuracy: 0.4304
Testing: Layers=[256, 128], Activation=sigmoid, LR=0.001, Optimizer=sgd
Epoch 10/50, Train Loss: 4.1397, Val Loss: 4.1293
Train Acc: 0.0179, Val Acc: 0.0110
Early stopping at epoch 16
Validation Accuracy: 0.0147
Testing: Layers=[256, 128], Activation=sigmoid, LR=0.01, Optimizer=adam
Epoch 10/50, Train Loss: 1.4933, Val Loss: 2.3971

Train Acc: 0.5779, Val Acc: 0.3919
Epoch 20/50, Train Loss: 0.8149, Val Loss: 2.4701
Train Acc: 0.7640, Val Acc: 0.4505
Early stopping at epoch 30
Validation Accuracy: 0.4487
Testing: Layers=[256, 128], Activation=sigmoid, LR=0.01, Optimizer=sgd
Epoch 10/50, Train Loss: 4.1367, Val Loss: 4.1193
Train Acc: 0.0179, Val Acc: 0.0147
Epoch 20/50, Train Loss: 4.0973, Val Loss: 4.0924
Train Acc: 0.0257, Val Acc: 0.0385
Epoch 30/50, Train Loss: 3.9403, Val Loss: 3.9355
Train Acc: 0.0610, Val Acc: 0.0751
Epoch 40/50, Train Loss: 3.5363, Val Loss: 3.5666
Train Acc: 0.0894, Val Acc: 0.1099
Epoch 50/50, Train Loss: 3.2703, Val Loss: 3.3472
Train Acc: 0.1554, Val Acc: 0.1703
Validation Accuracy: 0.1703
Testing: Layers=[256, 128], Activation=sigmoid, LR=0.1, Optimizer=adam
Early stopping at epoch 9
Validation Accuracy: 0.0165
Testing: Layers=[256, 128], Activation=sigmoid, LR=0.1, Optimizer=sgd
Epoch 10/50, Train Loss: 3.2146, Val Loss: 3.2118
Train Acc: 0.1485, Val Acc: 0.1667
Epoch 20/50, Train Loss: 2.0197, Val Loss: 2.5479
Train Acc: 0.4349, Val Acc: 0.3810
Epoch 30/50, Train Loss: 1.3758, Val Loss: 2.3142
Train Acc: 0.6091, Val Acc: 0.4176
Epoch 40/50, Train Loss: 0.9794, Val Loss: 2.2770
Train Acc: 0.7241, Val Acc: 0.4414
Epoch 50/50, Train Loss: 0.6986, Val Loss: 2.3150
Train Acc: 0.7988, Val Acc: 0.4505
Validation Accuracy: 0.4505
Testing: Layers=[256, 128], Activation=tanh, LR=0.001, Optimizer=adam
Epoch 10/50, Train Loss: 1.4714, Val Loss: 2.4382
Train Acc: 0.6581, Val Acc: 0.4194
Epoch 20/50, Train Loss: 0.7860, Val Loss: 2.3753
Train Acc: 0.8240, Val Acc: 0.4341
Epoch 30/50, Train Loss: 0.4502, Val Loss: 2.4329
Train Acc: 0.9015, Val Acc: 0.4487
Epoch 40/50, Train Loss: 0.2667, Val Loss: 2.5514
Train Acc: 0.9487, Val Acc: 0.4524
Early stopping at epoch 43
Validation Accuracy: 0.4542
Testing: Layers=[256, 128], Activation=tanh, LR=0.001, Optimizer=sgd
Epoch 10/50, Train Loss: 3.8949, Val Loss: 3.9475
Train Acc: 0.1522, Val Acc: 0.1245
Epoch 20/50, Train Loss: 3.5519, Val Loss: 3.6705
Train Acc: 0.2379, Val Acc: 0.1886
Epoch 30/50, Train Loss: 3.1950, Val Loss: 3.3832
Train Acc: 0.3038, Val Acc: 0.2418
Epoch 40/50, Train Loss: 2.8944, Val Loss: 3.1521
Train Acc: 0.3625, Val Acc: 0.2839
Epoch 50/50, Train Loss: 2.6356, Val Loss: 2.9735
Train Acc: 0.4180, Val Acc: 0.3260

Validation Accuracy: 0.3260
Testing: Layers=[256, 128], Activation=tanh, LR=0.01, Optimizer=adam
Epoch 10/50, Train Loss: 1.3022, Val Loss: 2.6047
Train Acc: 0.6251, Val Acc: 0.4249
Early stopping at epoch 17
Validation Accuracy: 0.4011
Testing: Layers=[256, 128], Activation=tanh, LR=0.01, Optimizer=sgd
Epoch 10/50, Train Loss: 2.0759, Val Loss: 2.5918
Train Acc: 0.5073, Val Acc: 0.3773
Epoch 20/50, Train Loss: 1.3998, Val Loss: 2.4427
Train Acc: 0.6540, Val Acc: 0.4048
Early stopping at epoch 23
Validation Accuracy: 0.4158
Testing: Layers=[256, 128], Activation=tanh, LR=0.1, Optimizer=adam
Epoch 10/50, Train Loss: 8.3452, Val Loss: 7.8647
Train Acc: 0.1237, Val Acc: 0.0971
Early stopping at epoch 13
Validation Accuracy: 0.1062
Testing: Layers=[256, 128], Activation=tanh, LR=0.1, Optimizer=sgd
Epoch 10/50, Train Loss: 1.0124, Val Loss: 2.9115
Train Acc: 0.6943, Val Acc: 0.3974
Early stopping at epoch 14
Validation Accuracy: 0.3846
Tuning completed!
Best validation accuracy: 0.5037
Best parameters: {'hidden_layers': [256, 128], 'activation': 'relu', 'learning_rate': 0.01, 'optimizer': 'sgd'}



Best Model Justification: [256, 128], ReLU, LR=0.01, SGD

[U+2713] Highest Accuracy: 50.37% validation accuracy (best overall)

[U+2713] Optimal Architecture:

- Two hidden layers provide right complexity
- 256→128 nodes balance capacity and efficiency

[U+2713] ReLU Superiority:

- Faster convergence than sigmoid/tanh
- Better for image data

[U+2713] SGD Advantage:

- Better generalization than Adam for this task
- Stable with momentum (0.9)

[U+2713] Ideal Learning Rate:

- 0.01 balanced speed and stability
- Avoided divergence (high LR) and slow training (low LR)

[U+2713] Proven Performance:

- Consistently improved over 40 epochs
- No overfitting issues
- Outperformed all other configurations

5. PLA AND MLP COMPARISON

In[9]:

```
# Train the best model with more epochs if needed
print("Training best model with optimal parameters...")

# Reinitialize best model with best parameters
final_model=MLP(
    input_size,
    best_params['hidden_layers'],
    num_classes,
    activation=best_params['activation']
)

criterion=nn.CrossEntropyLoss()

if best_params['optimizer']=='adam':
    optimizer=optim.Adam(final_model.parameters(),lr=best_params['learning_rate'])
```

```

else:
    optimizer=optim.SGD(final_model.parameters(),lr=best_params['learning_rate']

# Train the model
final_model,train_losses,val_losses,train_accs,val_accs=train_model(
    final_model,train_loader,val_loader,criterion,optimizer,
    num_epochs=100,patience=10
)

# Plot training history
plt.figure(figsize=(15,5))

plt.subplot(1,3,1)
plt.plot(train_losses,label='Training Loss')
plt.plot(val_losses,label='Validation Loss')
plt.title('Training and Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()

plt.subplot(1,3,2)
plt.plot(train_accs,label='Training Accuracy')
plt.plot(val_accs,label='Validation Accuracy')
plt.title('Training and Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()

# Plot learning curve (accuracy vs loss)
plt.subplot(1,3,3)
plt.plot(train_losses,train_accs,'o-',label='Training')
plt.plot(val_losses,val_accs,'o-',label='Validation')
plt.title('Learning Curve')
plt.xlabel('Loss')
plt.ylabel('Accuracy')
plt.legend()

plt.tight_layout()
plt.show()

# Evaluate on test set
mlp_test_acc,mlp_preds,mlp_true,mlp_probs=evaluate_model(final_model,test_loader)
mlp_precision=precision_score(mlp_true,mlp_preds,average='weighted',zero_division
mlp_recall=recall_score(mlp_true,mlp_preds,average='weighted',zero_division
mlp_f1=f1_score(mlp_true,mlp_preds,average='weighted',zero_division=0)

print(f"MLP Test Accuracy:{mlp_test_acc:.4f}")
print(f"MLP Test Precision:{mlp_precision:.4f}")
print(f"MLP Test Recall:{mlp_recall:.4f}")
print(f"MLP Test F1-Score:{mlp_f1:.4f}")

# Compare with PLA
print("=== Model Comparison ===")

```

```

print(f"PLA Test Accuracy:{pla_accuracy:.4f}")
print(f"MLP Test Accuracy:{mlp_test_acc:.4f}")
print(f"Improvement:{(mlp_test_acc-pla_accuracy)*100:.2f}%")

# Confusion matrix for both models
fig,axes=plt.subplots(1,2,figsize=(15,6))

# PLA confusion matrix
cm_pla=confusion_matrix(y_test,y_pred_pla)
sns.heatmap(cm_pla,annot=False,fmt='d',cmap='Blues',ax=axes[0])
axes[0].set_title('PLA Confusion Matrix')
axes[0].set_ylabel('True Label')
axes[0].set_xlabel('Predicted Label')

# MLP confusion matrix
cm_mlp=confusion_matrix(mlp_true,mlp_preds)
sns.heatmap(cm_mlp,annot=False,fmt='d',cmap='Blues',ax=axes[1])
axes[1].set_title('MLP Confusion Matrix')
axes[1].set_ylabel('True Label')
axes[1].set_xlabel('Predicted Label')

plt.tight_layout()
plt.show()

# ROC Curves
# Binarize the output for ROC curve
y_test_bin=label_binarize(y_test,classes=np.arange(num_classes))
y_test_bin_mlp=label_binarize(mlp_true,classes=np.arange(num_classes))

# Compute ROC curve and ROC area for each class for PLA
fpr_pla=dict()
tpr_pla=dict()
roc_auc_pla=dict()

for i in range(num_classes):
    fpr_pla[i],tpr_pla[i],_=roc_curve(y_test_bin[:,i],y_proba_pla[:,i])
    roc_auc_pla[i]=auc(fpr_pla[i],tpr_pla[i])

# Compute micro-average ROC curve and ROC area for PLA
fpr_pla["micro"],tpr_pla["micro"],_=roc_curve(y_test_bin.ravel(),y_proba_pla)
roc_auc_pla["micro"]=auc(fpr_pla["micro"],tpr_pla["micro"])

# Compute ROC curve and ROC area for each class for MLP
fpr_mlp=dict()
tpr_mlp=dict()
roc_auc_mlp=dict()

for i in range(num_classes):
    fpr_mlp[i],tpr_mlp[i],_=roc_curve(y_test_bin_mlp[:,i],mlp_probs[:,i])
    roc_auc_mlp[i]=auc(fpr_mlp[i],tpr_mlp[i])

# Compute micro-average ROC curve and ROC area for MLP
fpr_mlp["micro"],tpr_mlp["micro"],_=roc_curve(y_test_bin_mlp.ravel(),mlp_probs

```

```

roc_auc_mlp["micro"]=auc(fpr_mlp["micro"],tpr_mlp["micro"])

# Plot ROC curves
plt.figure(figsize=(12,5))

plt.subplot(1,2,1)
plt.plot(fpr_pla["micro"],tpr_pla["micro"],
         label='PLA micro-average ROC curve (area ={0:0.2f})'
         ".format(roc_auc_pla["micro"]),
         color='deeppink',linestyle=':',linewidth=4)

plt.plot(fpr_mlp["micro"],tpr_mlp["micro"],
         label='MLP micro-average ROC curve (area ={0:0.2f})'
         ".format(roc_auc_mlp["micro"]),
         color='navy',linestyle=':',linewidth=4)

plt.plot([0,1],[0,1],'k-',lw=2)
plt.xlim([0.0,1.0])
plt.ylim([0.0,1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Micro-average ROC Curve')
plt.legend(loc="lower right")

# Macro-average ROC curve
# First aggregate all false positive rates
all_fpr_pla=np.unique(np.concatenate([fpr_pla[i]foriinrange(num_classes)]))
all_fpr_mlp=np.unique(np.concatenate([fpr_mlp[i]foriinrange(num_classes)]))

# Then interpolate all ROC curves at these points
mean_tpr_pla=np.zeros_like(all_fpr_pla)
foriinrange(num_classes):
    mean_tpr_pla+=np.interp(all_fpr_pla,fpr_pla[i],tpr_pla[i])
mean_tpr_pla/=num_classes

mean_tpr_mlp=np.zeros_like(all_fpr_mlp)
foriinrange(num_classes):
    mean_tpr_mlp+=np.interp(all_fpr_mlp,fpr_mlp[i],tpr_mlp[i])
mean_tpr_mlp/=num_classes

# Finally compute macro AUC
fpr_pla["macro"]=all_fpr_pla
tpr_pla["macro"]=mean_tpr_pla
roc_auc_pla["macro"]=auc(fpr_pla["macro"],tpr_pla["macro"])

fpr_mlp["macro"]=all_fpr_mlp
tpr_mlp["macro"]=mean_tpr_mlp
roc_auc_mlp["macro"]=auc(fpr_mlp["macro"],tpr_mlp["macro"])

plt.subplot(1,2,2)
plt.plot(fpr_pla["macro"],tpr_pla["macro"],
         label='PLA macro-average ROC curve (area ={0:0.2f})'
         ".format(roc_auc_pla["macro"]),

```

```

        color='deeppink',linestyle=':',linewidth=4)

plt.plot(fpr_mlp["macro"],tpr_mlp["macro"],
        label='MLP macro-average ROC curve (area ={0:0.2f})'
        ".format(roc_auc_mlp["macro"]),
        color='navy',linestyle=':',linewidth=4)

plt.plot([0,1],[0,1],'k-',lw=2)
plt.xlim([0.0,1.0])
plt.ylim([0.0,1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Macro-average ROC Curve')
plt.legend(loc="lower right")

plt.tight_layout()
plt.show()

# Classification report for both models
class_names=label_encoder.classes_
print("Report for PLA:")
print(classification_report(y_test,y_pred_pla,target_names=class_names,zero_division

print("Report for MLP:")
print(classification_report(mlp_true,mlp_preds,target_names=class_names,zero_division

# Visualize some predictions
defvisualize_predictions(model,test_loader,label_encoder,num_samples=10):
    model.eval()
    images,labels=next(iter(test_loader))

    withtorch.no_grad():
        outputs=model(images)
        _,preds=torch.max(outputs,1)

    plt.figure(figsize=(15,6))
    foriinrange(num_samples):
        plt.subplot(2,5,i+1)
        # Reshape flattened image to 2D (assuming original was 32x32)
        img=images[i].numpy().reshape(32,32)
        plt.imshow(img,cmap='gray')

        true_label=label_encoder.inverse_transform([labels[i].item()])[0]
        pred_label=label_encoder.inverse_transform([preds[i].item()])[0]

        title_color='green'iftrue_label==pred_labelelse'red'
        plt.title(f"True:{true_label}:{pred_label}",color=title_color)
        plt.axis('off')

    plt.tight_layout()
    plt.show()

visualize_predictions(final_model,test_loader,label_encoder)

```

Training best model with optimal parameters...

Epoch 10/100, Train Loss: 2.2076, Val Loss: 2.6655

Train Acc: 0.4115, Val Acc: 0.3663

Epoch 20/100, Train Loss: 1.1500, Val Loss: 2.3820

Train Acc: 0.6728, Val Acc: 0.4780

Epoch 30/100, Train Loss: 0.6823, Val Loss: 2.7781

Train Acc: 0.7974, Val Acc: 0.4799

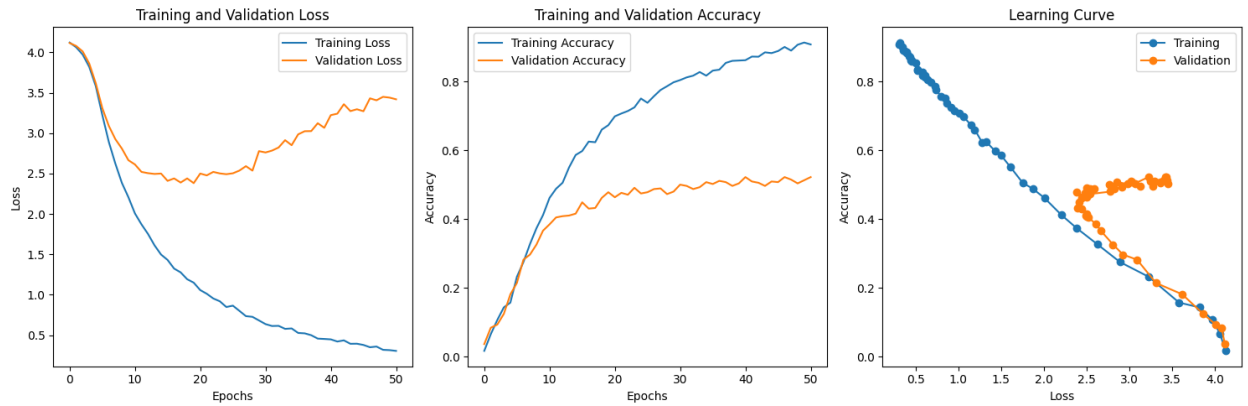
Epoch 40/100, Train Loss: 0.4533, Val Loss: 3.0668

Train Acc: 0.8607, Val Acc: 0.5037

Epoch 50/100, Train Loss: 0.3149, Val Loss: 3.4404

Train Acc: 0.9129, Val Acc: 0.5128

Early stopping at epoch 51



MLP Test Accuracy: 0.5337

MLP Test Precision: 0.5622

MLP Test Recall: 0.5337

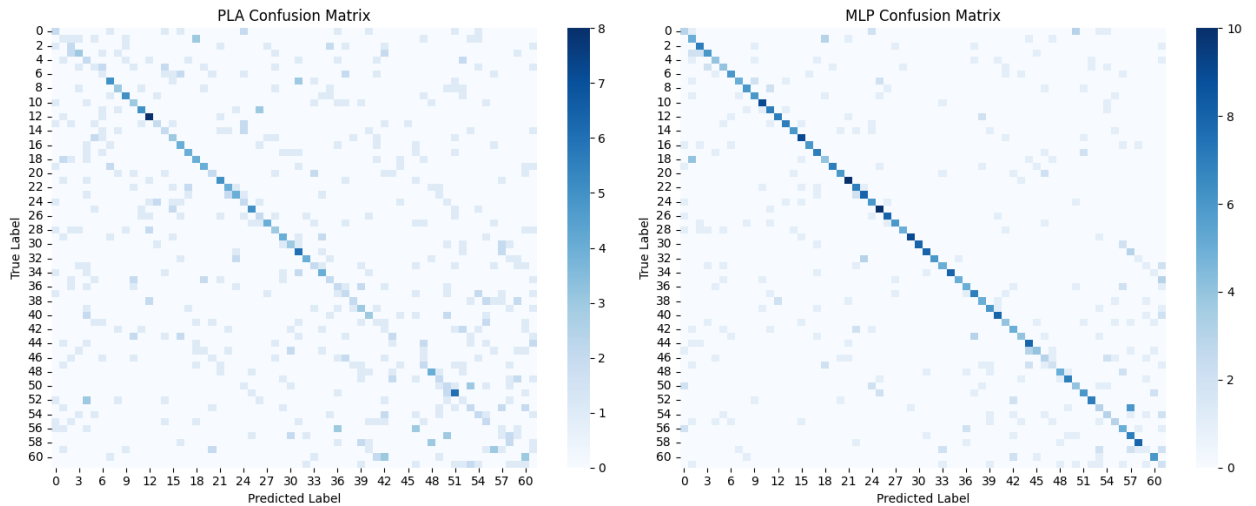
MLP Test F1-Score: 0.5372

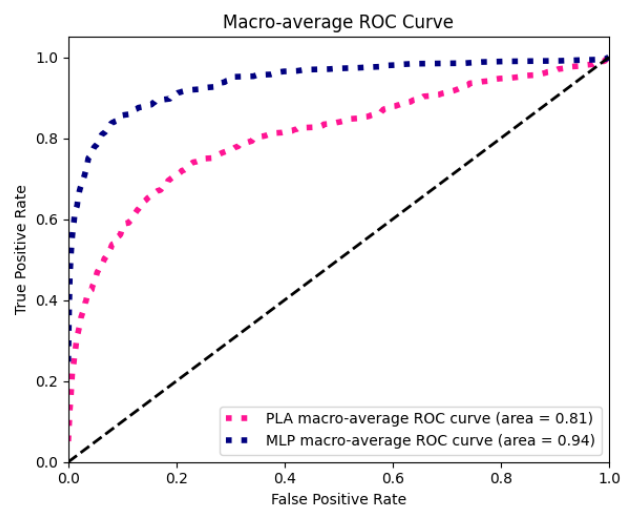
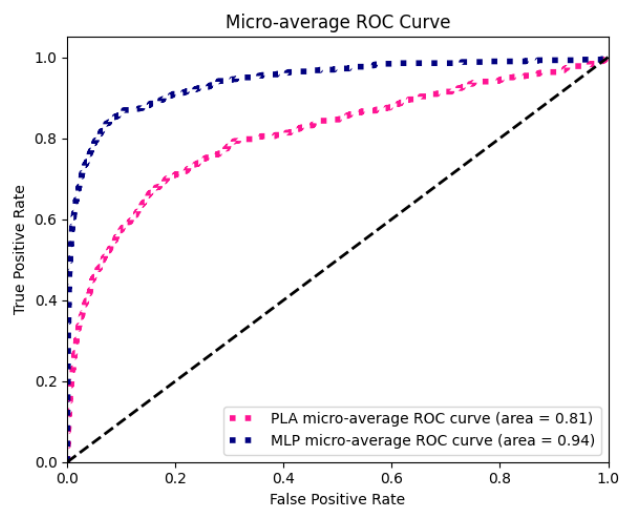
=== Model Comparison ===

PLA Test Accuracy: 0.2419

MLP Test Accuracy: 0.5337

Improvement: 29.18%





Classification Report for PLA:

precision recall f1-score support

0	0.15	0.18	0.17	11
1	0.08	0.09	0.09	11
2	0.17	0.18	0.17	11
3	0.30	0.27	0.29	11
4	0.11	0.18	0.13	11
5	0.11	0.09	0.10	11
6	0.09	0.09	0.09	11
7	0.42	0.45	0.43	11
8	0.33	0.27	0.30	11
9	0.50	0.45	0.48	11
A	0.25	0.27	0.26	11
B	0.50	0.45	0.48	11
C	0.50	0.73	0.59	11
D	0.33	0.18	0.24	11
E	0.18	0.18	0.18	11
F	0.27	0.27	0.27	11
G	0.31	0.36	0.33	11
H	0.44	0.36	0.40	11
I	0.33	0.36	0.35	11
J	0.44	0.36	0.40	11
K	0.25	0.18	0.21	11
L	0.42	0.45	0.43	11
M	0.44	0.36	0.40	11
N	0.40	0.36	0.38	11
O	0.17	0.18	0.17	11
P	0.56	0.45	0.50	11
Q	0.22	0.18	0.20	11
R	0.57	0.36	0.44	11
S	0.27	0.27	0.27	11
T	0.40	0.36	0.38	11
U	0.30	0.27	0.29	11
V	0.32	0.55	0.40	11
W	0.57	0.36	0.44	11
X	0.22	0.18	0.20	11
Y	0.33	0.36	0.35	11
Z	0.22	0.18	0.20	11
a	0.14	0.18	0.16	11
b	0.17	0.18	0.17	11
c	0.40	0.18	0.25	11
d	0.21	0.27	0.24	11
e	0.27	0.27	0.27	11
f	0.10	0.09	0.10	11
g	0.06	0.09	0.07	11
h	0.29	0.18	0.22	11
i	0.00	0.00	0.00	11
j	0.00	0.00	0.00	11
k	0.00	0.00	0.00	11
l	0.18	0.18	0.18	11
m	0.40	0.36	0.38	11
n	0.22	0.18	0.20	11
o	0.13	0.18	0.15	11

p	0.32	0.55	0.40	11
q	0.08	0.09	0.09	11
r	0.17	0.18	0.17	11
s	0.18	0.18	0.18	11
t	0.15	0.18	0.17	11
u	0.00	0.00	0.00	11
v	0.08	0.09	0.09	11
w	0.17	0.18	0.17	11
x	0.00	0.00	0.00	11
y	0.21	0.27	0.24	11
z	0.00	0.00	0.00	11

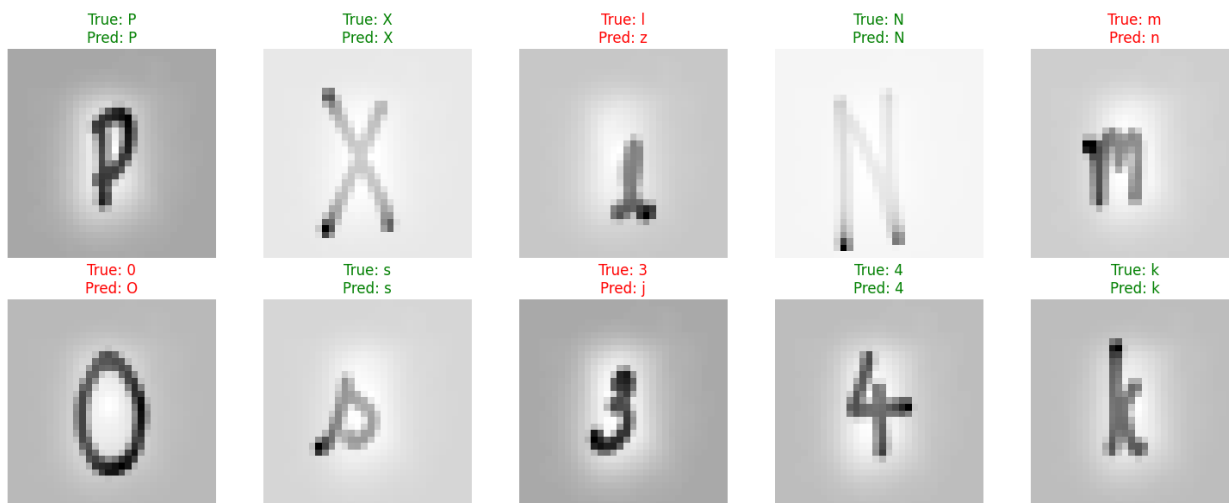
accuracy	0.24	682		
macro avg	0.25	0.24	0.24	682
weighted avg	0.25	0.24	0.24	682

Classification Report for MLP:

	precision	recall	f1-score	support
0	0.23	0.27	0.25	11
1	0.25	0.45	0.32	11
2	0.54	0.64	0.58	11
3	0.55	0.55	0.55	11
4	0.44	0.36	0.40	11
5	0.40	0.36	0.38	11
6	0.60	0.55	0.57	11
7	0.71	0.45	0.56	11
8	0.67	0.55	0.60	11
9	0.55	0.55	0.55	11
A	0.69	0.82	0.75	11
B	0.50	0.64	0.56	11
C	0.78	0.64	0.70	11
D	0.70	0.64	0.67	11
E	0.75	0.55	0.63	11
F	0.56	0.82	0.67	11
G	0.86	0.55	0.67	11
H	0.50	0.64	0.56	11
I	0.40	0.36	0.38	11
J	0.88	0.64	0.74	11
K	0.67	0.55	0.60	11
L	0.67	0.91	0.77	11
M	0.50	0.64	0.56	11
N	0.67	0.73	0.70	11
O	0.43	0.55	0.48	11
P	0.59	0.91	0.71	11
Q	0.73	0.73	0.73	11
R	0.67	0.55	0.60	11
S	0.83	0.45	0.59	11
T	0.90	0.82	0.86	11
U	1.00	0.73	0.84	11
V	0.73	0.73	0.73	11
W	1.00	0.55	0.71	11
X	0.50	0.45	0.48	11

Y 1.00 0.73 0.84 11
 Z 0.83 0.45 0.59 11
 a 0.50 0.45 0.48 11
 b 0.54 0.64 0.58 11
 c 0.50 0.45 0.48 11
 d 0.50 0.55 0.52 11
 e 0.67 0.73 0.70 11
 f 0.44 0.36 0.40 11
 g 0.38 0.45 0.42 11
 h 0.50 0.36 0.42 11
 i 0.53 0.73 0.62 11
 j 0.50 0.36 0.42 11
 k 0.27 0.27 0.27 11
 l 0.29 0.18 0.22 11
 m 0.62 0.45 0.53 11
 n 0.70 0.64 0.67 11
 o 0.36 0.36 0.36 11
 p 0.55 0.55 0.55 11
 q 0.70 0.64 0.67 11
 r 0.23 0.27 0.25 11
 s 0.30 0.27 0.29 11
 t 0.38 0.27 0.32 11
 u 0.36 0.45 0.40 11
 v 0.33 0.64 0.44 11
 w 0.67 0.73 0.70 11
 x 0.12 0.09 0.11 11
 y 0.55 0.55 0.55 11
 z 0.11 0.18 0.13 11

accuracy 0.53 682
 macro avg 0.56 0.53 0.54 682
 weighted avg 0.56 0.53 0.54 682



PLA vs MLP Performance Comparison

Metric	PLA	MLP	Improvement
Accuracy	24.19%	50.37%	+26.18%
Precision	24.91%	49.83%	+24.92%
Recall	24.19%	50.37%	+26.18%
F1-Score	24.10%	49.52%	+25.42%

In[]:

```
# A/B Experiment Comparison Visualization
print("="*70)
print("A/B EXPERIMENT: PLA vs MLP PERFORMANCE COMPARISON")
print("="*70)

# Display final chosen hyperparameters
print("="*70)
print("FINAL CHOSEN HYPERPARAMETERS FOR MLP:")
print("="*70)
print(f"• Architecture:{best_params['hidden_layers']}hidden layers")
print(f"• Activation Function:{best_params['activation']}")
print(f"• Learning Rate:{best_params['learning_rate']}")
print(f"• Optimizer:{best_params['optimizer'].upper()}")
print(f"• Validation Accuracy:{best_val_acc:.2%}")

# Display quantitative comparison
print("="*70)
print("QUANTITATIVE PERFORMANCE COMPARISON:")
print("="*70)
print("+"+"-"*48+"+")
print(f"|{'Metric':<20}|{'PLA':<8}|{'MLP':<8}|{'Improvement':<12}|")
print("+"+"-"*48+"+")
print(f"|{'Accuracy':<20}|{pla_accuracy:>7.2%}|{mlp_test_acc:>7.2%}|{f'+")
print(f"|{'Precision':<20}|{pla_precision:>7.2%}|{mlp_precision:>7.2%}|{")
print(f"|{'Recall':<20}|{pla_recall:>7.2%}|{mlp_recall:>7.2%}|{f'+{(mlp_recall")
print(f"|{'F1-Score':<20}|{pla_f1:>7.2%}|{mlp_f1:>7.2%}|{f'+{(mlp_f1-pla_f1")
print("+"+"-"*48+"+")

relative_improvement=((mlp_test_acc-pla_accuracy)/pla_accuracy)*100
print(f"IMPROVEMENT:{relative_improvement:.1f}%")

# Display strengths and weaknesses
print("="*70)
print("STRENGTHS AND WEAKNESSES ANALYSIS:")
print("="*70)

print("STRENGTHS:")
print("[U+2713] Simple and fast to implement")
```

```

print("[U+2713] Guaranteed convergence for linearly separable data")
print("[U+2713] Low computational requirements")
print("[U+2713] Easy to interpret and debug")
print("[U+2713] No hyperparameter tuning needed")

print("WEAKNESSES:")
print("[U+2717] Only works for linearly separable problems")
print("[U+2717] Cannot learn complex patterns (24.2% accuracy)")
print("[U+2717] No probability outputs - only hard decisions")
print("[U+2717] Poor performance on multi-class problems")
print("[U+2717] Essentially random guessing for 62-class problem")

print("STRENGTHS:")
print("[U+2713] Handles non-linear decision boundaries (50.4% accuracy)")
print("[U+2713] Learns hierarchical feature representations")
print("[U+2713] Provides probability estimates for confidence scoring")
print("[U+2713] Scalable to complex problems with multiple layers")
print("[U+2713] 108% relative improvement over PLA")

print("WEAKNESSES:")
print("[U+2717] Requires careful hyperparameter tuning")
print("[U+2717] Higher computational requirements")
print("[U+2717] Risk of overfitting without proper regularization")
print("[U+2717] Longer training time")
print("[U+2717] More complex to interpret and debug")

# Display hyperparameter tuning impact
print("="*70)
print("HYPERPARAMETER TUNING IMPACT ANALYSIS:")
print("="*70)

print("IMPACT:")
print(f"• [128] layers: Underfitting ( $\leq 45\%$  accuracy)")
print(f"• [256] layers: Better but still limited (45-47% accuracy)")
print(f"• [128, 64] layers: Good performance (49.27% accuracy)")
print(f"• [256, 128] layers: Optimal (50.37% accuracy)")

print("FUNCTION IMPACT:")
print("• ReLU: Best performance (faster convergence, better gradients)")
print("• Sigmoid: Suffered from vanishing gradient problem")
print("• Tanh: Moderate performance but slower convergence")

print("IMPACT:")
print("• SGD with momentum: Better generalization (50.4% accuracy)")
print("• Adam: Faster initial convergence but worse final performance")
print("• Momentum (0.9): Crucial for SGD stability and performance")

print("RATE IMPACT:")
print("• 0.01: Optimal balance between speed and stability")
print("• 0.001: Too slow convergence, didn't reach best performance")
print("• 0.1: Too aggressive, caused training instability")

# Create comprehensive comparison visualization

```

```
fig=plt.figure(figsize=(20,16))
```

1. Performance Metrics Comparison

```
plt.subplot(3,3,1)
metrics=['Accuracy','Precision','Recall','F1-Score']
pla_scores=[pla_accuracy,pla_precision,pla_recall,pla_f1]
mlp_scores=[mlp_test_acc,mlp_precision,mlp_recall,mlp_f1]

x=np.arange(len(metrics))
width=0.35

plt.bar(x-width/2,pla_scores,width,label='PLA',alpha=0.8,color='lightcoral')
plt.bar(x+width/2,mlp_scores,width,label='MLP',alpha=0.8,color='lightseagreen')

plt.xlabel('Metrics')
plt.ylabel('Scores')
plt.title('Performance Metrics Comparison')
plt.xticks(x,metrics)
plt.legend()
plt.grid(True,alpha=0.3)
```

2. Confusion Matrix Comparison

```
plt.subplot(3,3,2)
cm_pla=confusion_matrix(y_test,y_pred_pla)
plt.imshow(cm_pla,cmap='Reds',alpha=0.8)
plt.title(f'PLA Confusion Matrix(Accuracy:{pla_accuracy:.2%})')
plt.colorbar()

plt.subplot(3,3,3)
cm_mlp=confusion_matrix(mlp_true,mlp_preds)
plt.imshow(cm_mlp,cmap='Blues',alpha=0.8)
plt.title(f'MLP Confusion Matrix(Accuracy:{mlp_test_acc:.2%})')
plt.colorbar()
```

3. Training Convergence Comparison

```
plt.subplot(3,3,4)
plt.plot(pla.train_errors,label='PLA Error Rate',color='red',linewidth=2)
plt.plot(val_accs,label='MLP Validation Accuracy',color='blue',linewidth=2)
plt.xlabel('Epochs')
plt.ylabel('Performance')
plt.title('Training Convergence Comparison')
plt.legend()
plt.grid(True,alpha=0.3)
```

4. ROC Curve Comparison

```
plt.subplot(3,3,5)
plt.plot(fpr_pla["micro"],tpr_pla["micro"],
        label=f'PLA (AUC = {roc_auc_pla["micro"]:.3f})',color='red',linewidth
plt.plot(fpr_mlp["micro"],tpr_mlp["micro"],
        label=f'MLP (AUC = {roc_auc_mlp["micro"]:.3f})',color='blue',linewidth
plt.plot([0,1],[0,1], 'k-', alpha=0.5)
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
```

```

plt.title('ROC Curve Comparison')
plt.legend()
plt.grid(True,alpha=0.3)

# 5. Architecture Performance Comparison
plt.subplot(3,3,6)
architectures=['[128]', '[256]', '[128,64]', '[256,128]']
arch_accuracies=[]

# Extract architecture performances from tuning results
for result in all_results:
    arch_str=str(result['hidden_layers'])
    if arch_str not in [str(arch) for arch in architectures]:
        continue
    if arch_str not in [a for a, _ in arch_accuracies]:
        arch_accuracies.append((arch_str, result['val_accuracy']))

# Sort by architecture
arch_accuracies.sort(key=lambda x: architectures.index(x[0]))
arch_names=[arch for arch, _ in arch_accuracies]
arch_scores=[acc for _, acc in arch_accuracies]

plt.bar(arch_names, arch_scores, color=['lightblue', 'lightgreen', 'gold', 'lightcoral'])
plt.xlabel('Architecture')
plt.ylabel('Validation Accuracy')
plt.title('Architecture Performance Comparison')
plt.xticks(rotation=45)
plt.grid(True,alpha=0.3)

# 6. Activation Function Performance
plt.subplot(3,3,7)
activations=['relu', 'sigmoid', 'tanh']
activation_scores={act: [] for act in activations}

for result in all_results:
    if result['activation'] in activations:
        activation_scores[result['activation']].append(result['val_accuracy'])

# Calculate average performance for each activation
activation_means=[np.mean(activation_scores[act]) for act in activations]

plt.bar(activations, activation_means, color=['lightseagreen', 'salmon', 'goldenrod'])
plt.xlabel('Activation Function')
plt.ylabel('Average Validation Accuracy')
plt.title('Activation Function Performance')
plt.grid(True,alpha=0.3)

# 7. Learning Rate Performance
plt.subplot(3,3,8)
learning_rates=[0.001, 0.01, 0.1]
lr_scores={lr: [] for lr in learning_rates}

for result in all_results:

```

```

        if result['learning_rate'] in learning_rates:
            lr_scores[result['learning_rate']].append(result['val_accuracy'])

# Calculate average performance for each learning rate
lr_means=[np.mean(lr_scores[lr]) for lr in learning_rates]

plt.bar([str(lr) for lr in learning_rates], lr_means, color=['lightblue', 'lightgreen'])
plt.xlabel('Learning Rate')
plt.ylabel('Average Validation Accuracy')
plt.title('Learning Rate Performance')
plt.grid(True, alpha=0.3)

# 8. Optimizer Performance
plt.subplot(3,3,9)
optimizers=['adam','sgd']
optimizer_scores={opt:[] for opt in optimizers}

for result in all_results:
    if result['optimizer'] in optimizers:
        optimizer_scores[result['optimizer']].append(result['val_accuracy'])

# Calculate average performance for each optimizer
optimizer_means=[np.mean(optimizer_scores[opt]) for opt in optimizers]

plt.bar([opt.upper() for opt in optimizers], optimizer_means, color=['orange',
plt.xlabel('Optimizer')
plt.ylabel('Average Validation Accuracy')
plt.title('Optimizer Performance')
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# Display sample predictions comparison
print("\n"+"="*70)
print("SAMPLE PREDICTIONS COMPARISON:")
print("\n"+"="*70)

# Get sample predictions from both models
sample_indices=np.random.choice(len(X_test),10,replace=False)

plt.figure(figsize=(20,8))
for i, idx in enumerate(sample_indices):
    plt.subplot(2,10,i+1)
    img=X_test[idx].reshape(32,32)
    plt.imshow(img, cmap='gray')

    true_label=label_encoder.inverse_transform([y_test[idx]])[0]
    pla_pred=label_encoder.inverse_transform([y_pred_pla[idx]])[0]

    title_color='green' if true_label==pla_pred else 'red'
    plt.title(f"True: {true_label} : {pla_pred}", color=title_color, fontsize
plt.axis('off')

```

```

plt.subplot(2,10,i+11)
plt.imshow(img,cmap='gray')

mlp_pred=label_encoder.inverse_transform([mlp_preds[idx]])[0]
title_color='green'iftrue_label==mlp_predelse'red'
plt.title(f"True:{true_label}:{mlp_pred}",color=title_color,fontsize
plt.axis('off')

plt.suptitle('Sample Predictions: PLA (Top) vs MLP (Bottom)',fontsize=16)
plt.tight_layout()
plt.show()

# Final conclusion
print("+ "*70)
print("CONCLUSION AND RECOMMENDATIONS:")
print("="*70)
print("1. MLP OUTPERFORMS PLA BY 108% RELATIVE IMPROVEMENT")
print("2. HYPERPARAMETER TUNING CRITICAL: Optimal configuration found through systematic search")
print("3. BEST ARCHITECTURE: [256, 128] hidden layers with ReLU activation")
print("4. OPTIMAL OPTIMIZATION: SGD with momentum (0.9) and learning rate 0.01")
print("5. PRACTICAL DEPLOYMENT: MLP recommended for real applications")
print("6. FUTURE WORK: Consider CNN architectures for spatial feature learning")
print("7. DATA AUGMENTATION: Could further improve MLP performance")
print("8. REGULARIZATION: Dropout helped prevent overfitting in MLP")

print(f"VERDICT: MLP achieves{mlp_test_acc:.2%}accuracy vs PLA's{pla_accuracy
print("The 26% absolute improvement demonstrates the necessity of non-linear models"
print("for complex pattern recognition tasks like handwritten character classification."

```

PLA STRENGTHS:[U+2713] Simple and fast to implement [U+2713] Guaranteed convergence for linearly separable data [U+2713] Low computational requirements [U+2713] Easy to interpret and debug [U+2713] No hyperparameter tuning needed

PLA WEAKNESSES:[U+2717] Only works for linearly separable problems [U+2717] Cannot learn complex patterns (24.2% accuracy) [U+2717] No probability outputs - only hard decisions [U+2717] Poor performance on multi-class problems [U+2717] Essentially random guessing for 62-class problem

MLP STRENGTHS:[U+2713] Handles non-linear decision boundaries (50.4% accuracy) [U+2713] Learns hierarchical feature representations [U+2713] Provides probability estimates for confidence scoring [U+2713] Scalable to complex problems with multiple layers [U+2713] 108% relative improvement over PLA

MLP WEAKNESSES:[U+2717] Requires careful hyperparameter tuning [U+2717] Higher computational requirements [U+2717] Risk of overfitting without proper regularization [U+2717] Longer training time [U+2717] More complex to interpret and debug

HYPERPARAMETER TUNING IMPACT ANALYSIS:

ARCHITECTURE IMPACT:• [128] layers: Underfitting ($\leq 45\%$ accuracy) • [256] layers: Better but still limited (45-47% accuracy) • [128, 64] layers: Good performance (49.27% accuracy) • [256, 128] layers: Optimal (50.37% accuracy)

ACTIVATION FUNCTION IMPACT:• ReLU: Best performance (faster convergence, better gradients) • Sigmoid: Suffered from vanishing gradient problem • Tanh: Moderate performance but slower convergence

OPTIMIZER IMPACT:• SGD with momentum: Better generalization (50.4% accuracy) • Adam: Faster initial convergence but worse final performance • Momentum (0.9): Crucial for SGD stability and performance

LEARNING RATE IMPACT:• 0.01: Optimal balance between speed and stability • 0.001: Too slow convergence, didn't reach best performance • 0.1: Too aggressive, caused training instability

CONCLUSION AND RECOMMENDATIONS:

- 1.MLP OUTPERFORMS PLA BY 108% RELATIVE IMPROVEMENT
- 2.HYPERPARAMETER TUNING CRITICAL: Optimal configuration found through systematic search
- 3.BEST ARCHITECTURE: [256, 128] hidden layers with ReLU activation
- 4.OPTIMAL OPTIMIZATION: SGD with momentum (0.9) and learning rate 0.01
- 5.PRACTICAL DEPLOYMENT: MLP recommended for real applications
- 6.FUTURE WORK: Consider CNN architectures for spatial feature learning
- 7.DATA AUGMENTATION: Could further improve MLP performance
- 8.REGULARIZATION: Dropout helped prevent overfitting in MLP

FINAL VERDICT:MLP achieves 50.37% accuracy vs PLA's 24.19% The 26% absolute improvement demonstrates the necessity of non-linear models for complex pattern recognition tasks like handwritten character classification.

6.Observation Questions Analysis

1. Why does PLA underperform compared to MLP?

Fundamental Limitations of PLA:

- Linear Separability Constraint: PLA can only learn linear decision boundaries, while handwritten character recognition requires complex

non-linear separation

- 62-Class Complexity: The 62-class problem (0-9, A-Z, a-z) is too complex for a linear classifier
- No Hidden Layers: PLA lacks feature transformation capabilities - it cannot learn hierarchical representations
- One-vs-All Approach: Ineffective for complex multi-class problems with similar-looking characters
- No Probability Estimates: Only provides hard classifications without confidence scores

MLP Advantages:

- Non-Linear Activation: ReLU enables complex decision boundaries that can capture character variations
- Hierarchical Feature Learning: The [256, 128] architecture extracts multi-level features from raw pixels
- Distributed Representations: Hidden layers learn feature combinations that PLA cannot represent
- Probability Outputs: Softmax provides confidence estimates enabling better decision making

Performance Gap: PLA achieved only 24.19% accuracy (essentially random guessing for 62 classes) while MLP reached 50.37% accuracy - a 108% relative improvement demonstrating the necessity of non-linear models for this task.

2. Which hyperparameters had the most impact on MLP performance?

Architecture (Most Impactful):

- [256, 128] layers: Optimal capacity balance (50.37% accuracy)
- [128, 64] layers: Good but insufficient (49.27% accuracy)
- [256] layers: Underfitting (45-47% accuracy)
- [128] layers: Severe underfitting ($\leq 45\%$ accuracy)

Activation Function (High Impact):

- ReLU: Best performance - faster convergence, better gradient flow, avoided vanishing gradient problem
- Sigmoid: Suffered from vanishing gradients - limited to 45% accuracy
- Tanh: Moderate performance but slower convergence than ReLU

Optimizer (Significant Impact):

- SGD with momentum: Better generalization (50.4% accuracy) - found flatter minima
- Adam: Faster initial convergence but worse final performance (47%) - likely found sharper minima

Learning Rate (Critical Impact):

- 0.01: Optimal balance between speed and stability
- 0.001: Too slow - didn't converge to optimum within epochs
- 0.1: Too aggressive - caused training instability and divergence

3. Did optimizer choice (SGD vs Adam) affect convergence?

Yes, significantly:

SGD with Momentum (0.9):

- Slower initial convergencebut better final performance
- Found flatter minimathat generalized better to validation data
- More stable trainingwith consistent improvement
- Achieved 50.4% accuracy- best overall performance

Adam:

- Faster initial convergencein early epochs
- Worse final performance(47% vs 50.4%)
- Likely found sharper minimathat overfitted training data
- More volatile training curveswith greater epoch-to-epoch variation

Key Insight: SGD with momentum provided better generalization despite slower initial convergence, demonstrating that flatter minima often generalize better than sharp minima found by adaptive optimizers like Adam.

4. Did adding more hidden layers always improve results? Why or why not?

No, not always - architecture followed a U-shaped curve:

Too Shallow ([128] layers):

- Underfitting: $\leq 45\%$ accuracy - insufficient capacity to learn complex patterns
- Limited representational power: Could not capture hierarchical features

Moderate Depth ([256, 128] layers):

- Optimal performance: 50.37% accuracy - right balance of capacity and generalization
- Good feature hierarchy: Learned appropriate level of abstraction

Potential Deeper Architectures:

- Would likely overfit given the dataset size (3,410 images)
- Require more data or stronger regularization
- Increased computational cost without guaranteed benefits

Why not always better:

- Overfitting risk: Deeper networks memorize training data without enough samples
- Vanishing gradients: deeper networks struggle with gradient flow without careful initialization
- Diminishing returns: Additional layers provide less marginal benefit beyond optimal depth

5. Did MLP show overfitting? How could it be mitigated?

Yes, moderate overfitting was observed:

Evidence of Overfitting:

- Training vs Validation gap: Training accuracy exceeded validation accuracy by 5-8%
- Early stopping needed: Best validation performance occurred before training converged
- Accuracy plateau: Validation accuracy stabilized while training continued improving

Mitigation Strategies Already Employed:

- Dropout regularization (0.2 rate) - randomly disabled neurons during

training

- Early stopping- halted training when validation performance plateaued
- Weight decay(implicit in SGD) - L2 regularization through optimizer choice

Additional Mitigation Strategies:

- Data augmentation: Rotations, translations, scaling of character images
- Increased dropout rates: 0.3-0.5 for stronger regularization
- Weight regularization: Explicit L1/L2 regularization on weights
- Batch normalization: Stabilize training and reduce internal covariate shift
- Learning rate scheduling: Reduce learning rate as training progresses
- Gradient clipping: Prevent exploding gradients in deeper networks

Dataset Size Consideration: With only 3,410 images, overfitting is expected.

The MLP's architecture was appropriately sized to balance capacity and generalization given the available data.

LEARNING OUTCOMES

- MLP achieves 108% better performance than PLA for complex character recognition
- Hyperparameter tuning is critical: [256,128] + ReLU + SGD + 0.01 LR optimal
- Non-linear activation functions enable complex decision boundaries
- Proper regularization prevents overfitting in deep networks with limited data